Comunicándonos con el usuario. Interfaces.

Caso práctico

Ana está cursando el módulo de Programación.

En el aula suele sentarse junto a su compañero **José Javier**.

En clase llevan unos días explicándoles cómo construir aplicaciones Java, utilizando interfaces gráficas de usuario (GUI).

Pero, ¿qué es una interfaz de usuario? A grandes rasgos, les han comentado que una interfaz de usuario es el medio con que el usuario puede comunicarse con una computadora. Las interfaces gráficas de usuario incluyen elementos tales como: menús, ventanas, paneles, etc., en general, elementos gráficos que facilitan la comunicación entre el ser humano y la computadora.

Hasta ahora, las aplicaciones de ejemplo que habían realizado, estaban en modo consola, o modo carácter, y están contentos porque están viendo las posibilidades que se les abren ahora. Están comprobando que podrán dar a sus aplicaciones un aspecto mucho más agradable para el usuario. **Ana** le comenta a **José Javier**:

—Así podremos dar un aspecto profesional a nuestros programas.

1. Introducción.

Hoy en día las **interfaces** de los programas son cada vez más sofisticadas y atractivas para el usuario. Son intuitivas y cada vez más fáciles de usar: pantallas táctiles, etc.

Sin embargo, no siempre ha sido así. No hace muchos años, antes de que surgieran y se popularizaran las interfaces gráficas de usuario para que el usuario interactuara con el sistema operativo con sistemas como Windows, etc., se trabajaba en **modo consola, o modo carácter,** es decir, se le daban las órdenes al ordenador con comandos por teclado, de hecho, por entonces no existía el ratón.

Así que, con el tiempo, con la idea de simplificar el uso de los ordenadores para extender el uso a un cada vez mayor espectro de gente, de usuarios de todo tipo, y no sólo para los expertos, se ha convertido en una práctica habitual utilizar **interfaces gráficas de usuario** (**IGU** ó **GUI** en inglés), para que el usuario interactúe y establezca un contacto más fácil e intuitivo con el ordenador.

En ocasiones verás otras definiciones de interfaz, como la que define una interfaz como un dispositivo que permite comunicar dos sistemas que no hablan el mismo lenguaje. También se emplea el término interfaz para definir el juego de conexiones y dispositivos que hacen posible la comunicación entre dos sistemas.

Aquí en este módulo, cuando hablamos de interfaz nos referimos a la cara visible de los programas tal y como se presenta a los usuarios para que interactúen con la máquina. La interfaz gráfica implica la presencia de un monitor de ordenador, en el que veremos la interfaz constituida por una serie de **menús e iconos que representan las opciones que el usuario** puede tomar dentro del sistema.

Las interfaces gráficas de usuario proporcionan al usuario ventanas, cuadros de diálogo, barras de herramientas, botones, listas desplegables y muchos otros elementos. Las aplicaciones son conducidas por eventos y se desarrollan haciendo uso de las clases que para ello nos ofrece el API de Java.

> **En 1981 aparecieron los primeros ordenadores personales, los llamados PC, pero hasta 1993 no se generalizaron las interfaces gráficas de usuario. El escritorio del sistema operativo Windows de Microsoft y su sistema de ventanas sobre la pantalla se ha estandarizado y universalizado, pero fueron los ordenadores Macintosh de la compañía Apple los pioneros en introducir las interfaces gráficas de usuario.**

Aquí un pequeño vídeo que resume la evolución de las GUI de los SO más reconocibles.

Autoevaluación

Señala la opción correcta acerca de las interfaces gráficas de usuario:

- ○ Son sinónimos de ficheros de texto.
- ○ Las ventanas de una aplicación no serían un ejemplo de elemento de interfaz gráfica de usuario.
- ○ Surgen con la idea de facilitar la interactuación del usuario con la máquina.
- ○ Ninguna es correcta.

2. Librerías Java para desarrollar GUIs.

Hemos visto que la interfaz gráfica de usuario es la parte del programa que permite al usuario interaccionar con él. Pero, ¿cómo la podemos crear en Java?

El API de Java proporciona una librería de clases para el desarrollo de interfaces gráficas de usuario (en realidad son dos: **AWT** y **Swing**). Esas librerías se engloban bajo los nombres de AWT y Swing, que a su vez forman parte de las **Java Foundation Classes** o JFC. Con estas librerías es como se han desarrollado tradicionalmente las interfaces gráficas en Java. En el año 2008, Oracle Corporations, lanza una serie de productos y tecnologías que bautizó como **JavaFX** para aportar nuevas funcionalidades a las interfaces gráficas de Java, adaptarlas a los nuevos vientos de tecnologías móviles, web, incorporación de CSS, etc, sobre la que fue añadiendo mejoras en sus sucesivas versiones.

En esta Unidad vamos a centrarnos fundamentalmente en JavaFX, en cómo construir proyectos con JavaFX y sus componentes con la herramienta gráfica **JavaFX Builder**, aunque daremos un breve repaso a AWT y también alguna indicación y recursos para trabajar con Swing.

2.1. AWT

Cuando apareció Java, los componentes gráficos disponibles para el desarrollo de GUI se encontraban en la librería denominada Abstract Window Toolkit (**AWT**).
Las clases AWT se desarrollaron usando código nativo (o sea, código asociado a una plataforma concreta). Eso **dificultaba la portabilidad** de las aplicaciones. Al usar código nativo, para poder conservar la portabilidad era necesario restringir la funcionalidad a los mínimos comunes a todas las plataformas donde se pretendía usar AWT. Como consecuencia, AWT es una librería con una funcionalidad muy pobre.

La estructura básica de la librería gira en torno a componentes y contenedores.
Los contenedores contienen componentes y son componentes a su vez, de forma que los eventos pueden tratarse tanto en contenedores como en componentes.

AWT es adecuada para interfaces gráficas sencillas, pero no para proyectos más complejos.
Más tarde, cuando apareció Java 2 surgió una librería más robusta, versátil y flexible: **Swing**. AWT aún sigue estando disponible, de hecho se usa por los componentes de Swing.

Puedes ver la lista de los principales componentes de la clase AWT en el siguiente enlace:

Principales componentes de la clase AWT

Para saber más

Página oficial de Sun – Oracle sobre AWT (en inglés).

AWT (Abstract Window Toolkit)

AWT sigue siendo imprescindible, ya que todos **los componentes Swing se construyen haciendo uso de clases de AWT**. De hecho, como puedes comprobar en el API, todos los componentes Swing, como por ejemplo `JButton` (es la clase Swing que usamos para crear cualquier botón de acción en una ventana), derivan de la clase `JComponent`, que a su vez deriva de la clase AWT `Container`.

Las clases asociadas a cada uno de los componentes AWT se encuentran en el paquete `java.awt`.
Las clases relacionadas con el manejo de eventos en AWT están en el paquete `java.awt.event`.

AWT fue la primera forma de construir las ventanas en Java, pero:

- limitaba la portabilidad,
- restringía la funcionalidad y
- requería demasiados recursos.

Autoevaluación

AWT está indicado para proyectos muy grandes y de gran complejidad.
Verdadero. ○ Falso. ○

Para saber más

Decíamos más arriba, que las JFC proporcionan soporte para impresión. En el siguiente enlace tienes más información sobre impresión en Java.

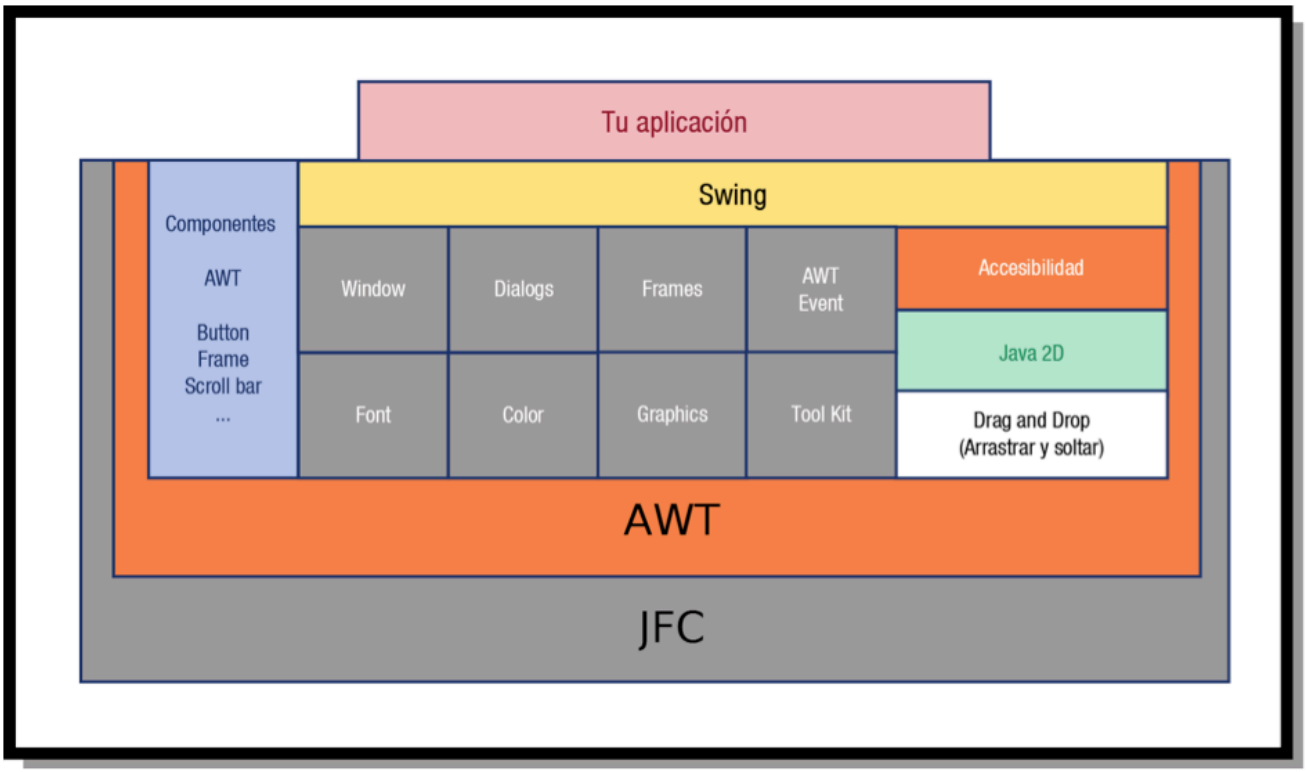
Impresión en Java

2.2. Swing

Cuando se vio que era necesario mejorar las características que ofrecía AWT, distintas empresas empezaron a sacar sus controles propios para mejorar algunas de las características de AWT. Así, Netscape sacó una librería de clases llamada **Internet Foundation Classes** para usar con Java, y eso obligó a Sun (todavía no adquirida por Oracle) a reaccionar para adaptar el lenguaje a las nuevas necesidades.

Se desarrolló en colaboración con Netscape todo el conjunto de componentes Swing que se añadieron a la JFC.

Swing es una librería de Java para la generación del GUI en aplicaciones. Swing se apoya sobre AWT y añade `JComponents`. La arquitectura de los componentes de Swing facilita la personalización de apariencia y comportamiento, si lo comparamos con los componentes AWT.



Por cada componente AWT (excepto `Canvas`) existe un componente Swing equivalente, cuyo nombre empieza por J, que permite más funcionalidad siendo menos pesado. Así, por ejemplo, para el componente AWT `Button` existe el equivalente Swing `JButton`, que permite como funcionalidad adicional la de crear botones con distintas formas (rectangulares, circulares, etc.), incluir imágenes en el botón, tener distintas representaciones para un mismo botón según esté seleccionado, o bajo el cursor, etc.

La razón por la que no existe `JCanvas` es que los paneles de la clase `JPanel` ya soportan todo lo que el componente `Canvas` de AWT soportaba. No se consideró necesario añadir un componente Swing `JCanvas` por separado.

Algunas características más de Swing, podemos mencionar:

- Es independiente de la arquitectura (metodología no nativa propia de Java)
- Proporciona todo lo necesario para la creación de entornos gráficos, tales como diseño de menús, botones, cuadros de texto, manipulación de eventos, etc**.**
- **Los componentes Swing no necesitan una ventana propia del sistema operativo cada uno**, sino que son visualizados dentro de la ventana que los contiene mediante métodos gráficos, por lo que requieren bastantes menos recursos.
- **Las clases Swing están completamente escritas en Java**, **con lo que la portabilidad es total**, a la vez que no hay obligación de restringir la funcionalidad a los mínimos comunes de todas las plataformas
- Por ello las clase Swing aportan una considerable gama de funciones que haciendo uso de la funcionalidad básica propia de AWT aumentan las posibilidades de diseño de interfaces gráficas.
- Debido a sus características, los componentes **AWT** se llaman componentes **"de peso pesado"** por la gran cantidad de recursos del sistema que usan, y los componentes **Swing** se llaman componentes **"de peso ligero"** por no necesitar su propia ventana del sistema operativo y por tanto consumir muchos menos recursos.

- Aunque todos los componentes Swing derivan de componentes AWT y de hecho se pueden mezclar en una misma aplicación componentes de ambos tipos, se desaconseja hacerlo. Es preferible desarrollar aplicaciones enteramente Swing, que requieren menos recursos y son más portables.

Para saber más

Si quieres conocer más ampliamente los componentes Swing, el editor gráfico Windows Builder y cómo hacer aplicaciones Java usando esta librería, te recomendamos esta serie de Videotutoriales.

2.3. JavaFX

JavaFX es una librería gráfica que pretende aportar mejoras a las librerías Swing y AWT.

Algunas de las características notables de JavaFX son:

- Java APIs: las APIs están escritas en código nativo Java compatibles con otros lenguajes soportados por la máquina virtual.
- **FXML** and **Scene Builder**: FXML es un lenguaje de marcado que describe las interfaces de usuario. Se pueden escribir directamente o usar la herramienta JavaFX Scene Builder para crearlos con una interfaz gráfica.
- **WebView**: permite embeber páginas HTML en las aplicaciones JavaFX. Ofrece soporte para JavaScript.
- **Built-in UI** controls and CSS: proporciona cantidad de controles para construir aplicaciones completas. El estilo de los controles puede ser modificado con CSS.
- **Canvas API**: para dibujar directamente en la pantalla.

El objetivo de esta unidad de interfaces gráficos y JavaFX es conocer una herramienta para crear interfaces gráficos más modernos que los que se pueden conseguir con Swing. No pretendemos ser expertos en desarrollo con JavaFX pero sí conocer esta tecnología de Java y manejar la herramienta **Scene Builder** para poder hacer interfaces gráficos más atractivos.

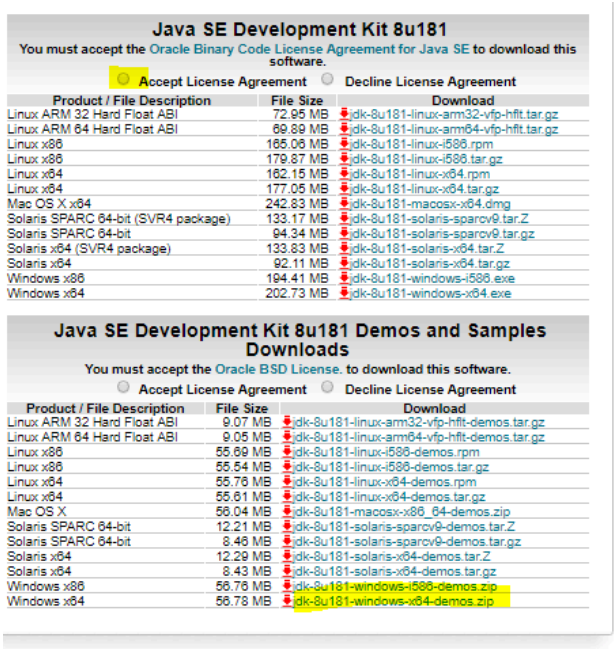
Debes conocer

Vamos a descargar ahora algunos de los ejemplos de JavaFX que suministra Oracle para ver todos los controles y opciones que vamos a poder manejar con JavaFX. El ejemplo se llama **Modena.Jar**

1. Accede a la descarga de ejemplos de Oracle para JavaFX



2. Presiona la opción Download y luego escoge la versión 8.181 de las Demos y Samples para tu versión de sistema operativo (en este caso Windows x64). ¡No olvides aceptar los acuerdos de la licencia!



Java SE Development Kit 8u181

You must accept the Oracle Binary Code License Agreement for Java SE to download this software.

Accept License Agreement Decline License Agreement

Product / File Description	File Size	Download
Linux ARM 32 Hard Float ABI	72.95 MB	jdk-8u181-linux-arm32-vfp-hflt.tar.gz
Linux ARM 64 Hard Float ABI	69.89 MB	jdk-8u181-linux-arm64-vfp-hflt.tar.gz
Linux x86	165.06 MB	jdk-8u181-linux-i586.rpm
Linux x86	179.87 MB	jdk-8u181-linux-i586.tar.gz
Linux x64	162.15 MB	jdk-8u181-linux-x64.rpm
Linux x64	177.05 MB	jdk-8u181-linux-x64.tar.gz
Mac OS X x64	242.83 MB	jdk-8u181-macosx-x64.dmg
Solaris SPARC 64-bit (SVR4 package)	133.17 MB	jdk-8u181-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	94.34 MB	jdk-8u181-solaris-sparcv9.tar.gz
Solaris x64 (SVR4 package)	133.83 MB	jdk-8u181-solaris-x64.tar.Z
Solaris x64	92.11 MB	jdk-8u181-solaris-x64.tar.gz
Windows x86	194.41 MB	jdk-8u181-windows-i586.exe
Windows x64	202.73 MB	jdk-8u181-windows-x64.exe

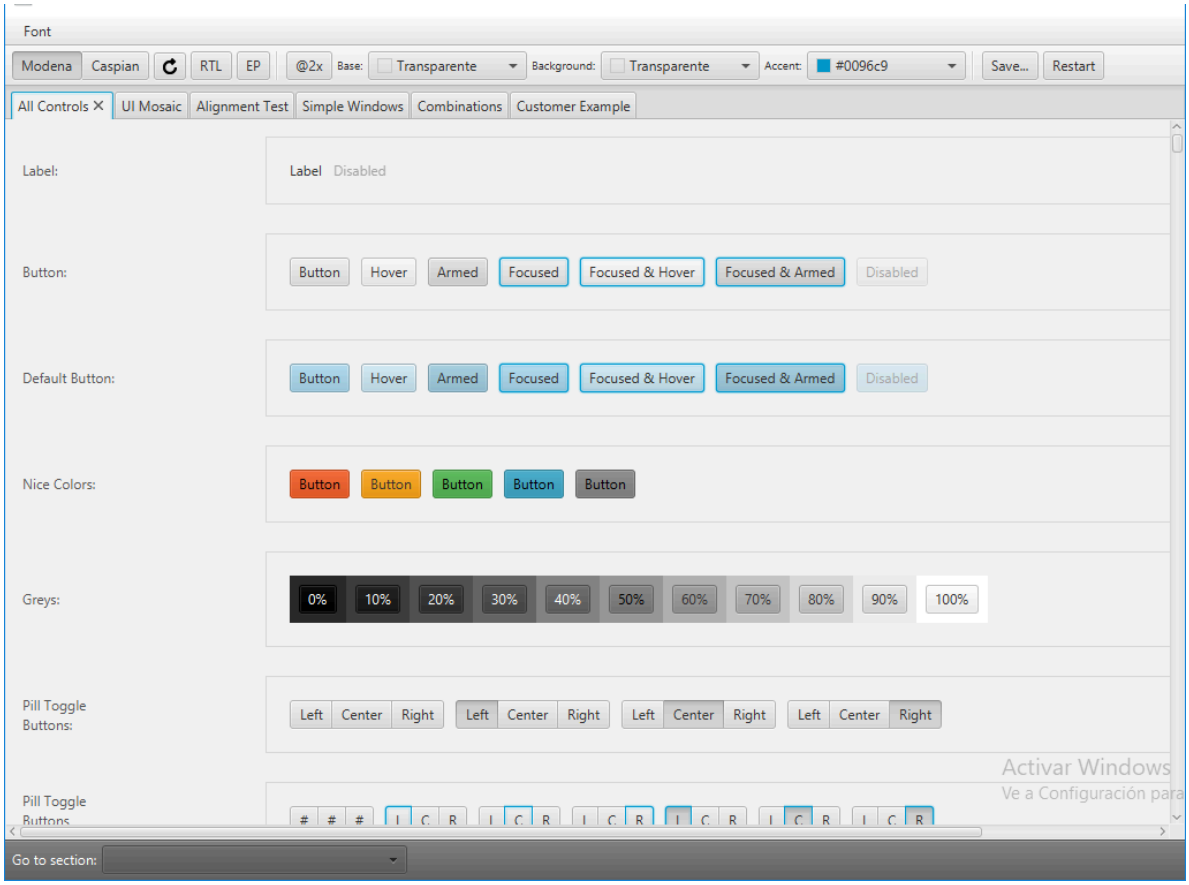
Java SE Development Kit 8u181 Demos and Samples Downloads

You must accept the Oracle BSD License. to download this software.

Accept License Agreement Decline License Agreement

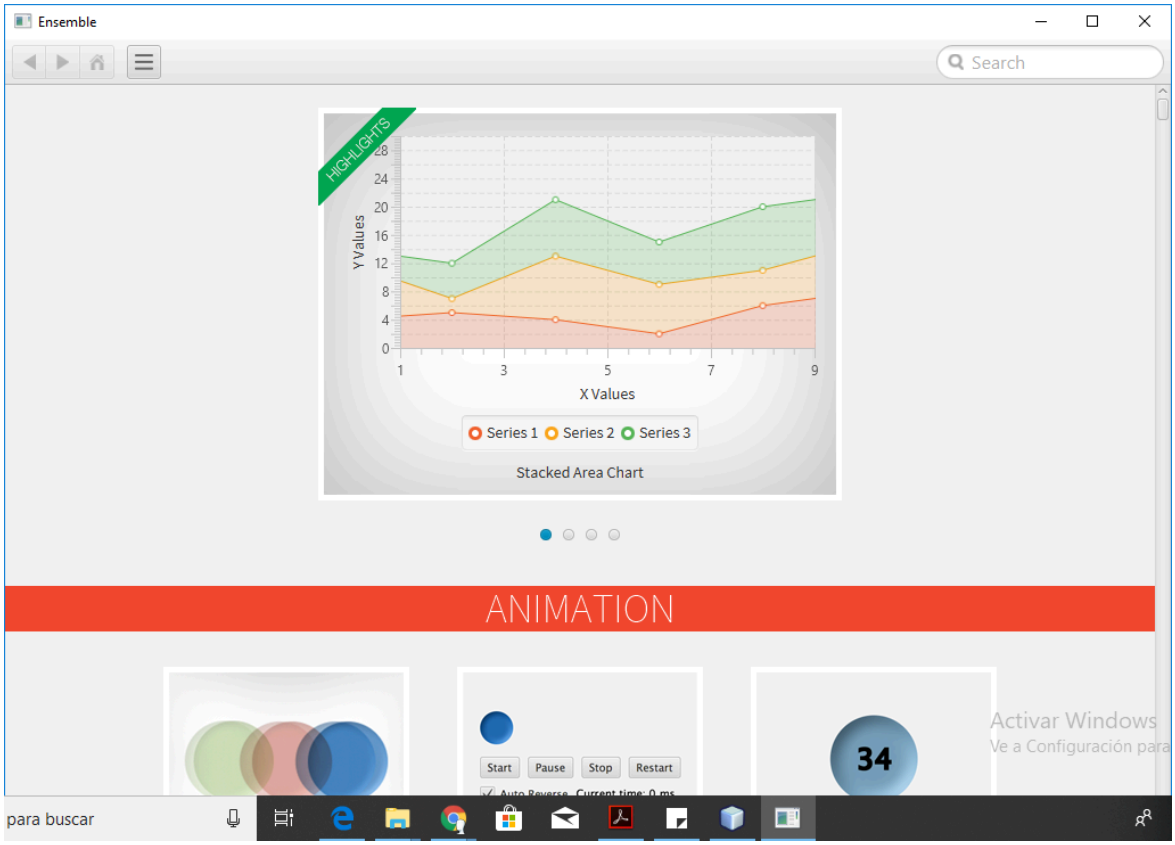
Product / File Description	File Size	Download
Linux ARM 32 Hard Float ABI	9.07 MB	jdk-8u181-linux-arm32-vfp-hflt-demos.tar.gz
Linux ARM 64 Hard Float ABI	9.05 MB	jdk-8u181-linux-arm64-vfp-hflt-demos.tar.gz
Linux x86	55.69 MB	jdk-8u181-linux-i586-demos.rpm
Linux x86	55.54 MB	jdk-8u181-linux-i586-demos.tar.gz
Linux x64	55.76 MB	jdk-8u181-linux-x64-demos.rpm
Linux x64	55.61 MB	jdk-8u181-linux-x64-demos.tar.gz
Mac OS X	56.04 MB	jdk-8u181-macosx-x86_64-demos.zip
Solaris SPARC 64-bit	12.21 MB	jdk-8u181-solaris-sparcv9-demos.tar.Z
Solaris SPARC 64-bit	8.46 MB	jdk-8u181-solaris-sparcv9-demos.tar.gz
Solaris x64	12.29 MB	jdk-8u181-solaris-x64-demos.tar.Z
Solaris x64	8.43 MB	jdk-8u181-solaris-x64-demos.tar.gz
Windows x86	56.76 MB	jdk-8u181-windows-i586-demos.zip
Windows x64	56.78 MB	jdk-8u181-windows-x64-demos.zip

3. Descomprime la carpeta y en la ruta \jdk1.8.0_181\demo\javafx_samples puedes encontrar mondena.jar ejecútalo con la JavaMV o con Javax.exe y se abrirá la aplicación de muestra con todos los opciones y controles que podrás programar. ¡Curiosea! !Investiga!



Para saber más

Si quieres, puedes ejecutar los otros ejemplos de JavaFX como **Ensemble8** o **3DViewer**. En esta unidad veremos fundamentalmente los controles para aplicaciones de escritorio y gestión. En la documentación de Oracle existen guías y tutoriales para dominar estos efectos.


Ensemble
Search
HIGHLIGHTS
Y Values
X Values
Series 1
Series 2
Series 3
Stacked Area Chart
ANIMATION
Start
Pause
Stop
Restart
34
Activar Windows
Ve a Configuración para
para buscar

3. Creación de interfaces gráficos de usuario utilizando asistentes y herramientas del entorno integrado.

Crear aplicaciones con interfaces gráficos JavaFX es relativamente sencillo con NetBeans. Este IDE nos permite crear fácilmente proyectos que incorporan esta tecnología para ir creando los componentes del interfaz gráfico por código.Un programador experimentado, probablemente no utilizará los asistentes, sino que escribirá, casi siempre, todo el código de la aplicación. De este modo, podrá indicar por código la ubicación de los diferentes controles, de su interfaz gráfica, en el contenedor (panel, marco, etc.) en el que se ubiquen.

Además de esta manera de codificar, existe una aplicación para crear los interfaces gráficos de nuestro proyecto y que se puede integrar fácilmente con NetBeans. Esta aplicación se llama **JavaFX Scene Builder** y la veremos en el subapartado siguiente.

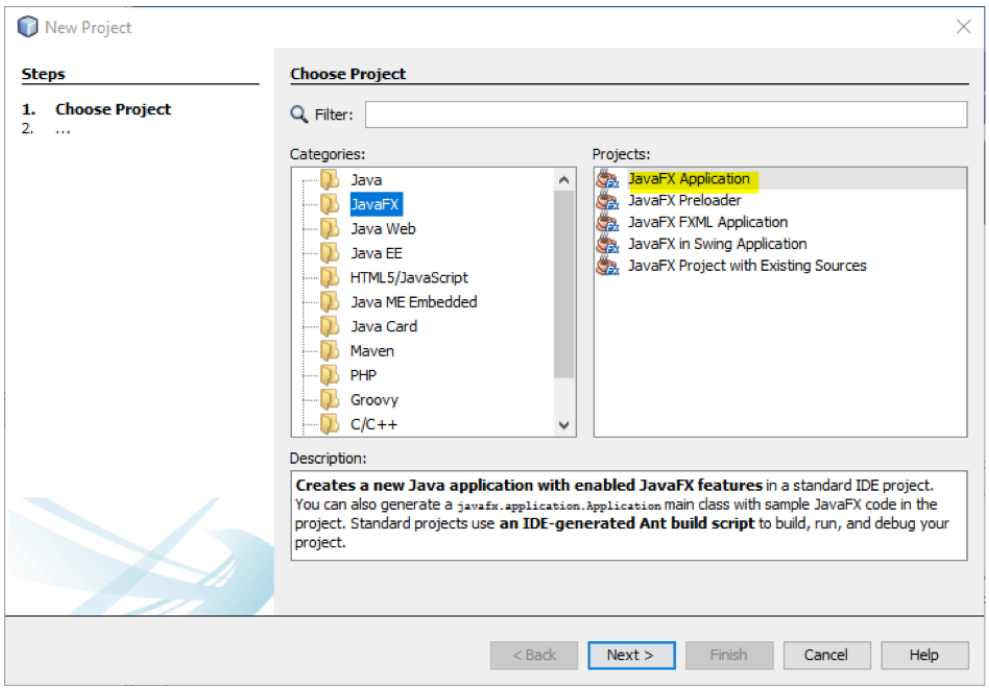
Los pasos para crear y ejecutar aplicaciones, se pueden resumir en:

- Crear un proyecto.
- Construir la interfaz con el usuario.
- Añadir funcionalidad, en base a la lógica de negocio que se requiera.
- Ejecutar el programa.

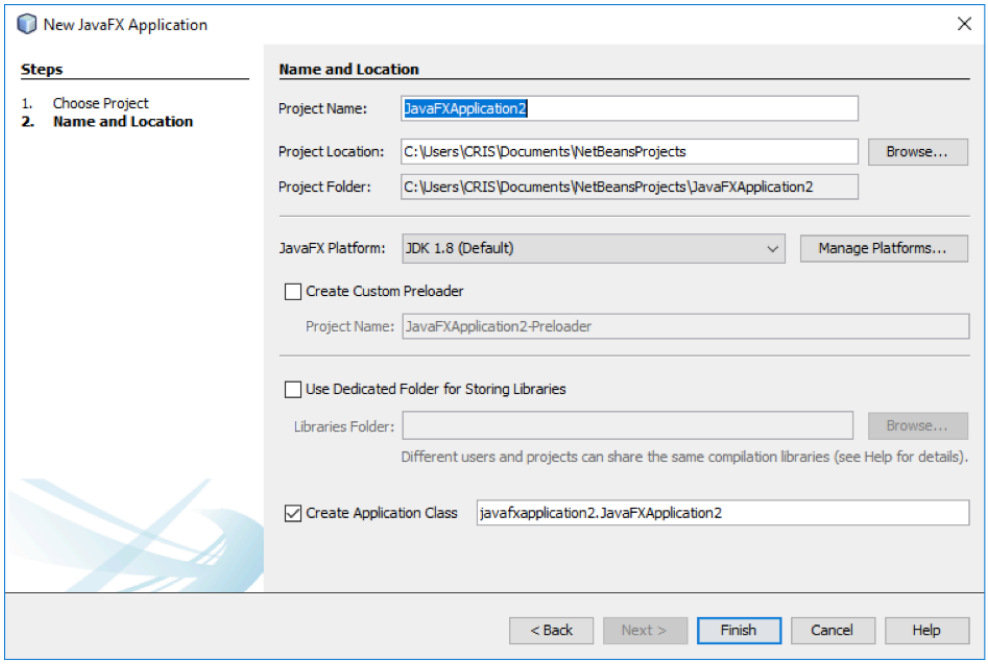
Debes conocer

Crear un proyecto JavaFX con entorno gráfico en NetBeans es relativamente sencillo. Recuerda que debemos tener como mínimo instalado NetBeans 8 y el JDK superior a la versión 1.8.80. Al comienzo del curso instalamos versiones que cumplían estos requisitos. Vamos a crear un proyecto por defecto, que nos servirá como primer “Hola Mundo” con interfaz gráfica y servirá para descubrir algunas de los aspectos que detallaremos en apartados siguientes como layout, componentes, controles, eventos, etc.

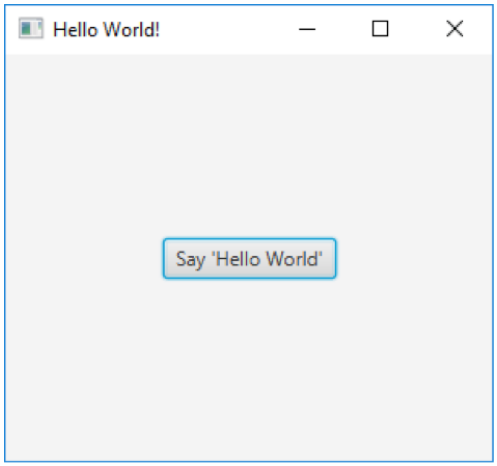
1. Al crear un Nuevo Proyecto, el asistente de NetBeans nos da la opción de crear un Proyecto JavaFX como JavaFX Application.



2. Al presionar Next nos pedirá el nombre del proyecto y se creará.



3. Con la clase que ha creado por defecto, tendremos una aplicación que genera un interfaz gráfico muy sencillo con un único botón, si se presiona, imprime por consola “Say ‘hello world”. Ejecuta el proyecto y pruébalo.



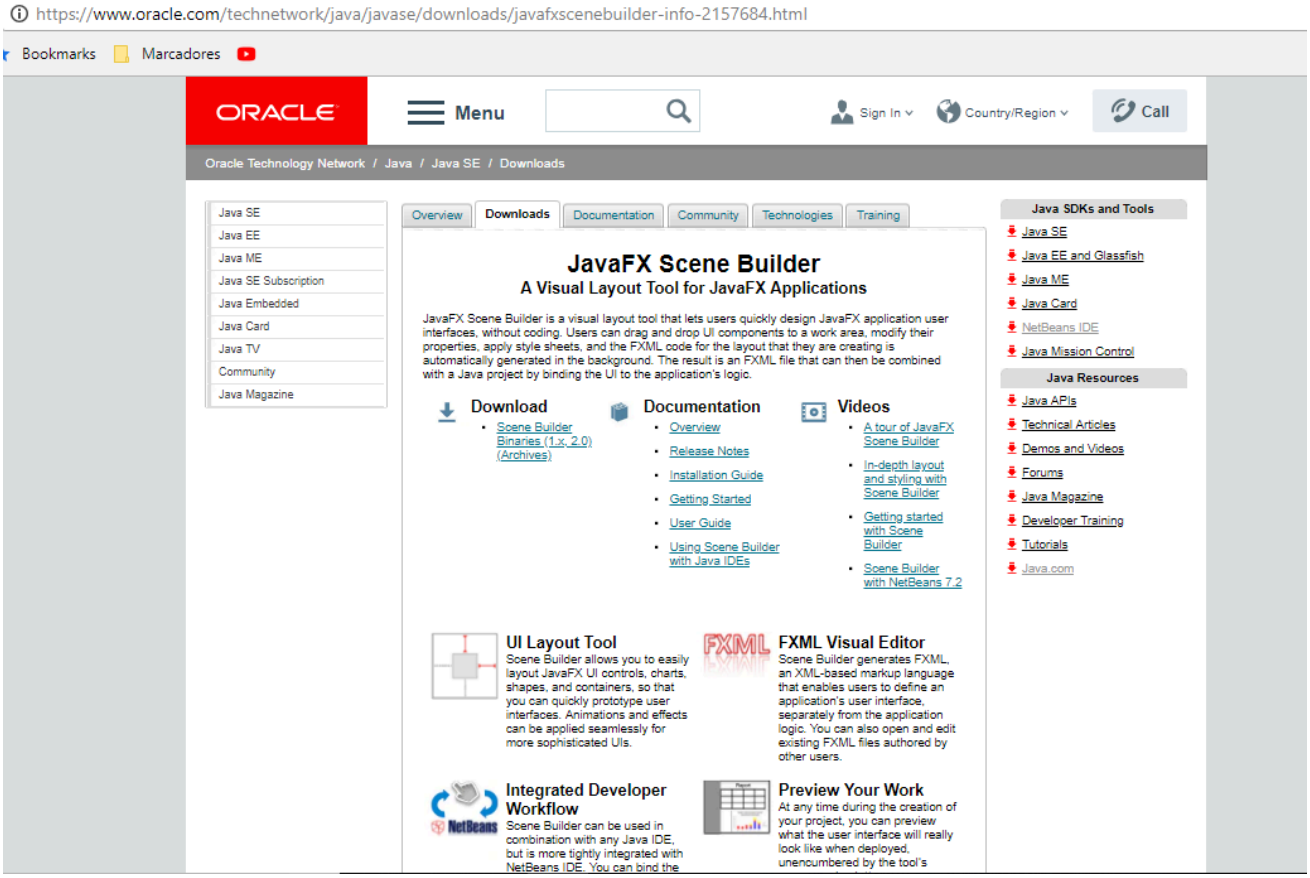
3.1. JavaFX Scene Builder

JavaFX Scene Builder es una herramienta visual que permite hacer rápidos diseños para interfaces gráficos en aplicaciones JavaFX, sin codificar. El programador podrá arrastrar los componentes gráficos a un área de trabajo, modificar sus propiedades, aplicar hoja de estilo CSS y el código **FXML** para el contenedor es creado automáticamente. El resultado es un archivo FXML que puede ser combinado en un proyecto Java con la lógica de la aplicación.

FXML es un lenguaje basado en XML diseñado para construir el interfaz gráfico de las aplicaciones JavaFX. FXML permite separar la construcción de la UI de la lógica del
programa. Un documento FXML será un fichero de texto (con la forma de un XML) con la extensión .fxml.

En cualquier caso, una escena JavaFX (scene) es una estructura jerárquica de objetos Java.

Vamos a **instalar JavaFX Scene Builder** y a dar alguna recomendación sobre él.
1. Accede al link de descarga de JavaFX Scene Information en la página de Oracle donde además existe mucha más información.



2. Presiona la opción de *Download* y después la versión 2.0 sobre el sistema operativo que utilices (Windows en nuestro caso). No olvides aceptar las condiciones de licencia.

JavaFX Scene Builder 1.1 Download

You must accept the Oracle Binary Code License Agreement for Java SE to download this software.

Accept License Agreement Decline License Agreement

Product / File Description	File Size	Download
Linux x86	54.04 MB	javafx_scenebuilder-1_1-linux-i586.deb
Linux x86	54.22 MB	javafx_scenebuilder-1_1-linux-i586.tar.gz
Linux x64	52.7 MB	javafx_scenebuilder-1_1-linux-x64.deb
Linux x64	52.88 MB	javafx_scenebuilder-1_1-linux-x64.tar.gz
MacOSX	59.92 MB	javafx_scenebuilder-1_1-macosx-universal.dmg
Windows	51.1 MB	javafx_scenebuilder-1_1-windows.msi
Samples	0.36 MB	javafx_scenebuilder_samples-1_1.zip

Back to top

JavaFX Scene Builder 2.0 Related Downloads

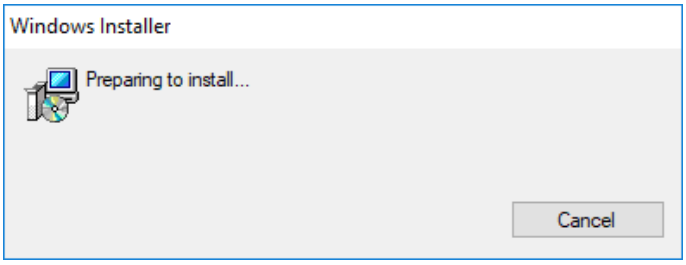
You must accept the Oracle BSD to download this software.

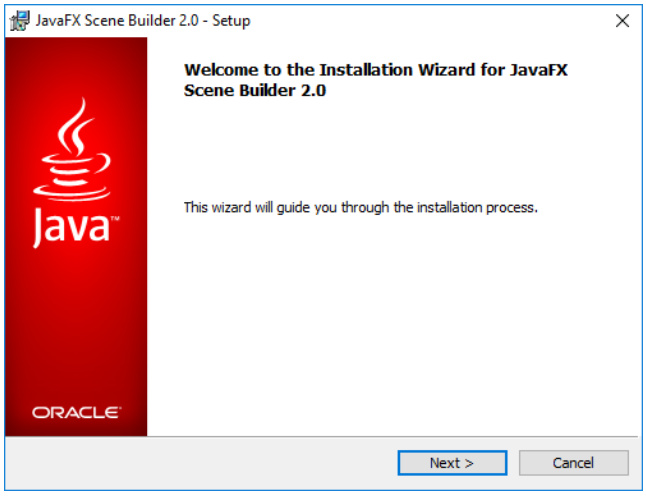
Accept License Agreement Decline License Agreement

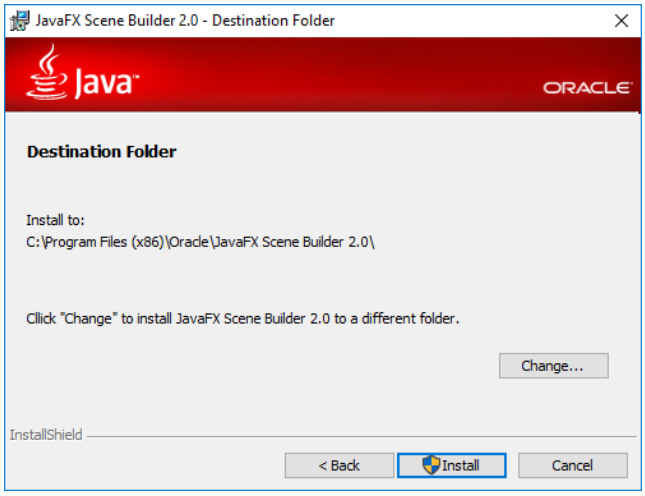
Product / File Description	File Size	Download
Windows 32/64 bit (msi)	56.1 MB	javafx_scenebuilder-2_0-windows.msi
Mac OS X (dmg)	68.6 MB	javafx_scenebuilder-2_0-macosx-universal.dmg
Linux 32-bit (deb)	61.5 MB	javafx_scenebuilder-2_0-linux-i586.deb
Linux 32-bit (tar.gz)	61.8 MB	javafx_scenebuilder-2_0-linux-i586.tar.gz
Linux 64-bit (tar.gz)	60.7 MB	javafx_scenebuilder-2_0-linux-x64.tar.gz
Linux 64-bit (deb)	60.5 MB	javafx_scenebuilder-2_0-linux-x64.deb
JavaFX Scene Builder 2.0 Samples	0.3 MB	javafx_scenebuilder_samples-2_0.zip
JavaFX Scene Builder 2.0 Kit API Documentation	1.2 MB	javafx_scenebuilder_kit_javadoc-2_0.zip
JavaFX Scene Builder 2.0 Kit Samples	68 KB	javafx_scenebuilder_kit_samples-2_0.zip

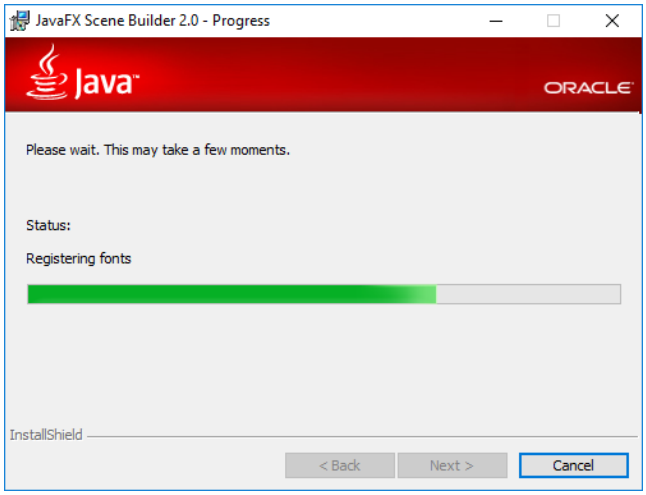
Back to top

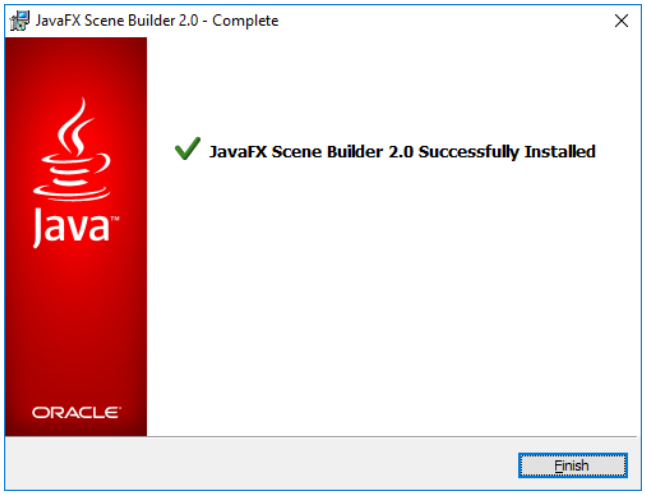
3. Cuando se haya descargado el paquete, procede con la instalación. Asigna la ubicación que desees.



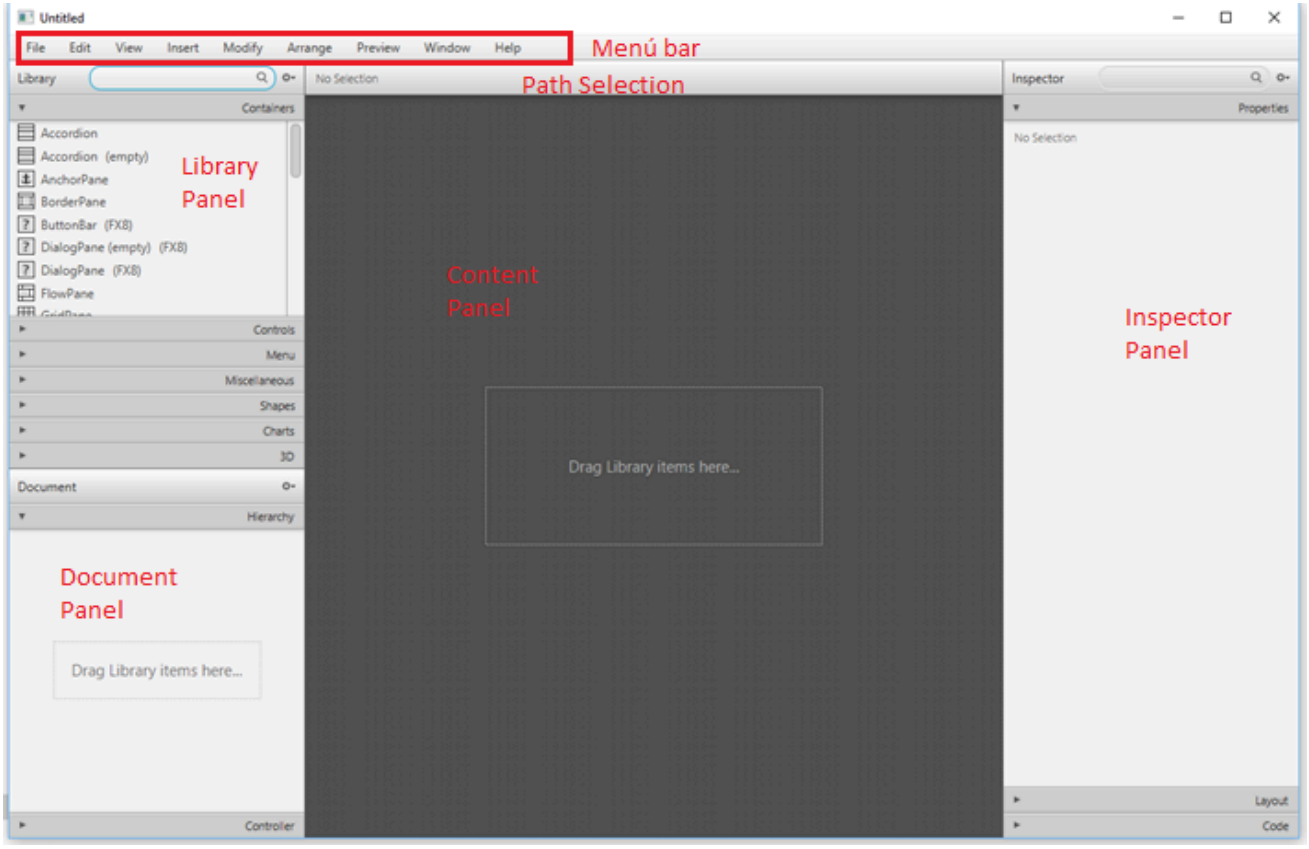








Una vez instalado, toca ejecutarlo. Aparecerá la siguiente aplicación



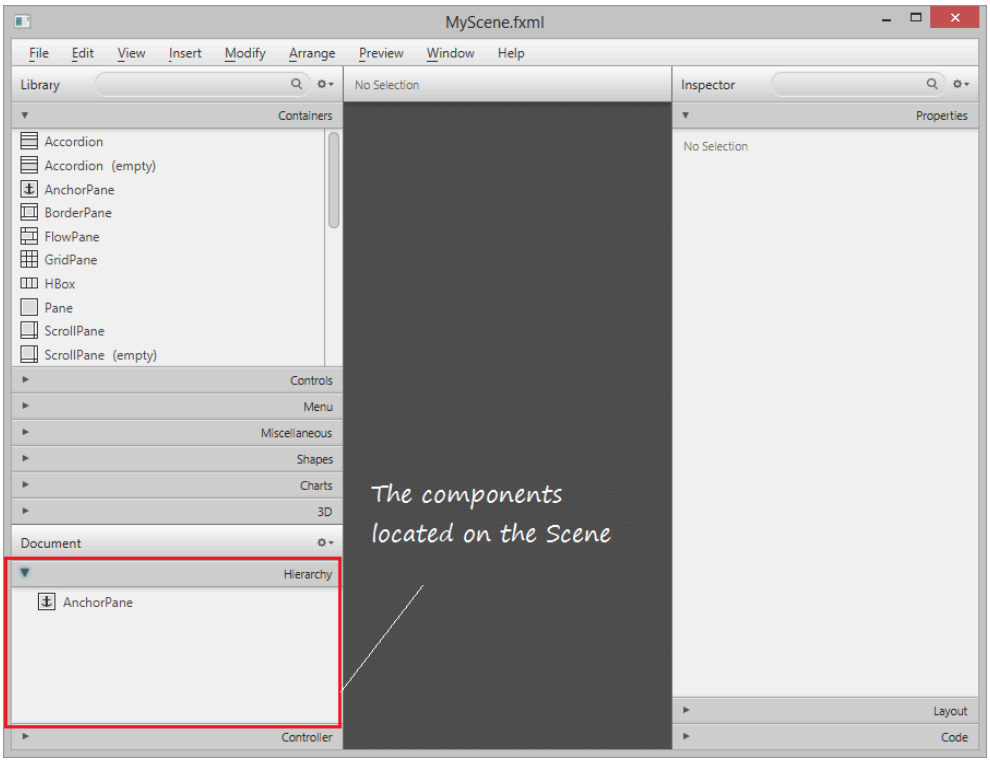
La ventana principal del JavaFX Scene Builder incluye las siguientes secciones:

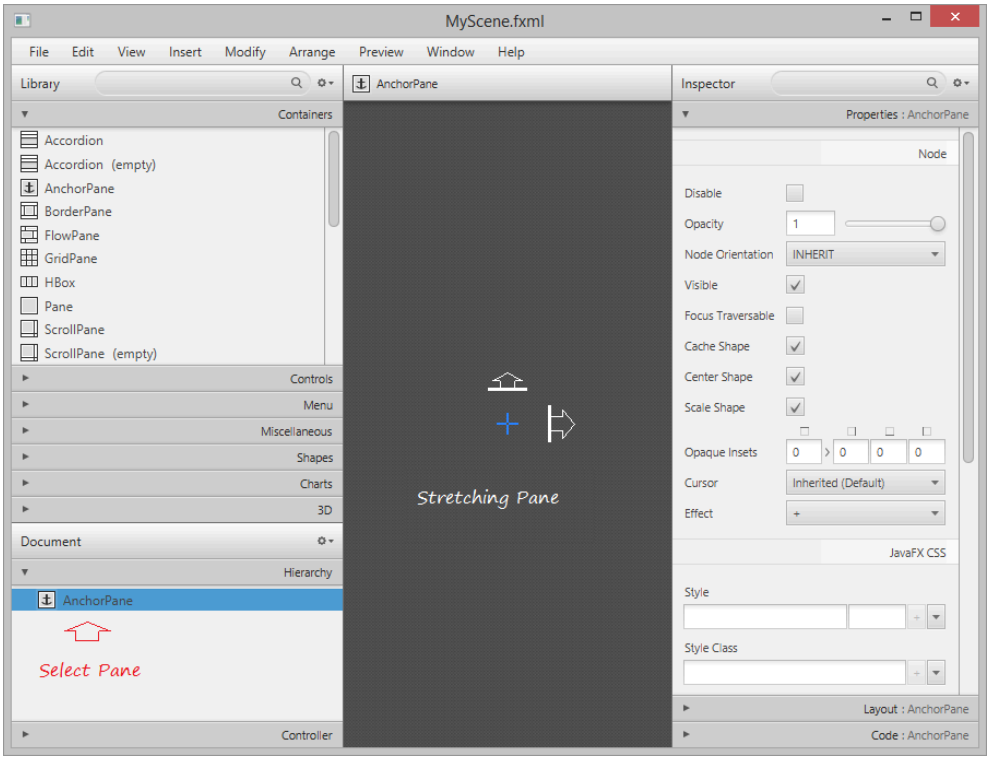
- **Menu Bar** proporciona acceso al menú de comandos.
- **Path, Selection and Message Bar** muestra el path del elemento seleccionado y permite escoger un element para poner el foco. También mostraría los mensajes de error o de status.
- **Content Panel** representa el scene container para los elementos del GUI que forman parte del FXML layout. Por defecto, un fichero FXML Nuevo tendrá un componente vacío cuando se abre JavaFX Scene Builder.
- **Library Panel** lista los elementos o controles JavaFX GUI disponibles para construir el FXML layout. Se selecciona de ahí para añadirlos al Content panel o Hierarchy panel.
- **Document Panel** contiene las secciones Hierarchy y Controller. La sección Hierarchy muestra un árbol con la representación del FXML layout que se está construyendo en el Content panel. La sección Controller permitirá gestionar los Fuentes del controlador e información sobre los valores asignados de fx:id.
- **Inspector Panel** contiene las propiedades, Layout, sección de Código que posibilita la gestión de los eventos utilizados por el emento GUI.

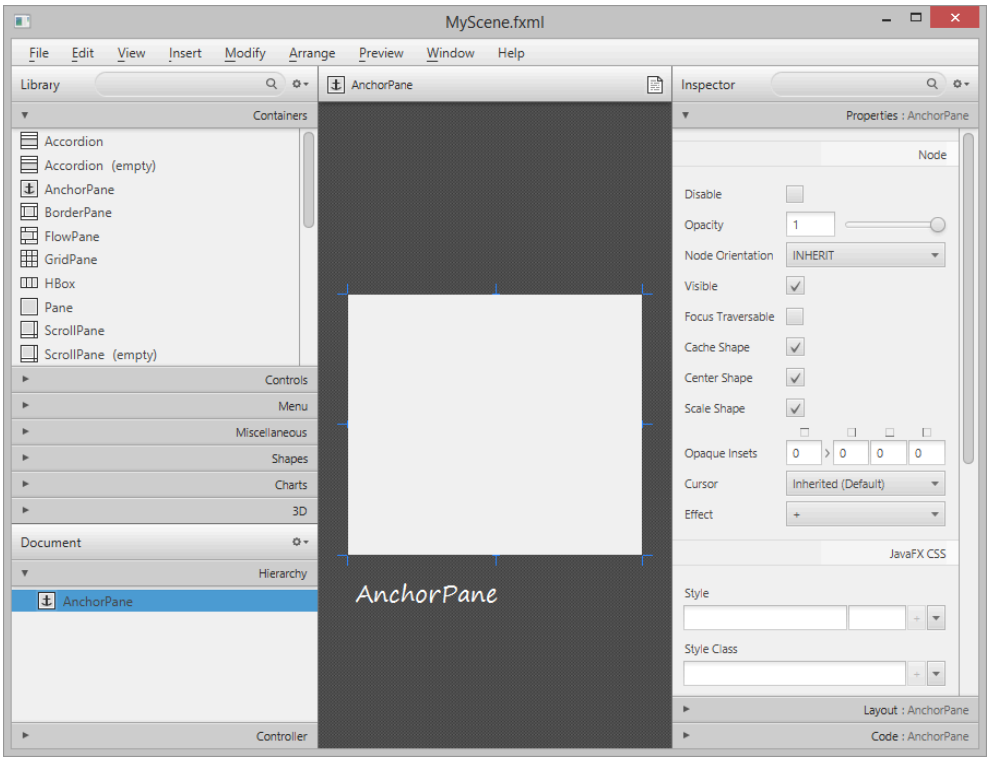
Debes conocer

Crear tus interfaces gráficos es muy sencillo con esta herramienta. Te dejamos aquí algunas indicaciones para realizar tus primeros UIs sencillas. En el siguiente subapartado, aprenderemos a integrarlas con NetBeans en un proyecto.

1. Vamos a crear el primer ejemplo de una aplicación JavaFX con una UI muy sencilla que tenga texto, un botón, etc. El elemento raíz del documento FXML es el objeto de más alto nivel en los objetos gráficos. En nuestro ejemplo será un AnchorPane (opción por defecto cuando se crea un FXML) [más adelante hablaremos de los tipos de Pane]

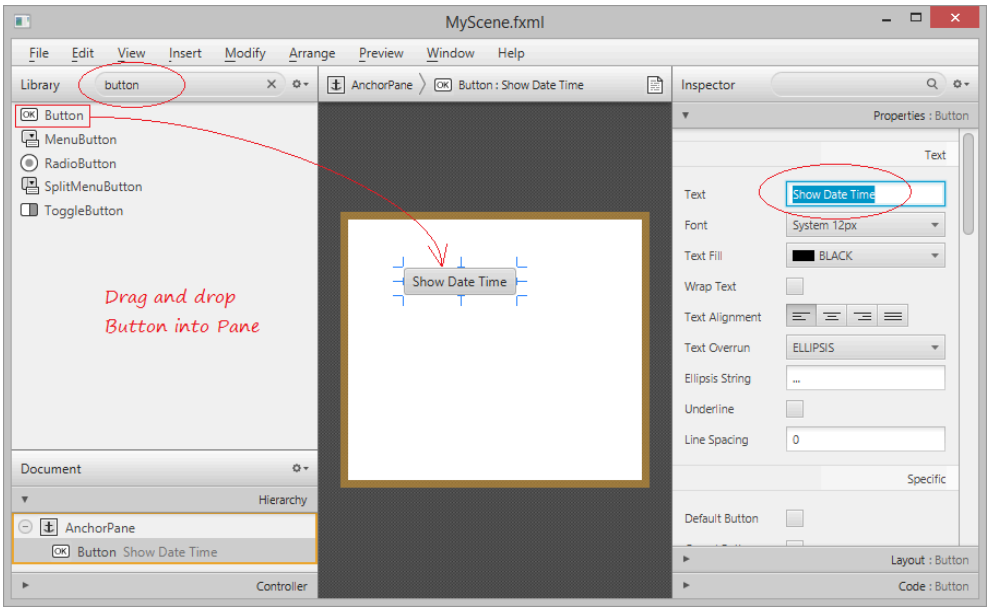






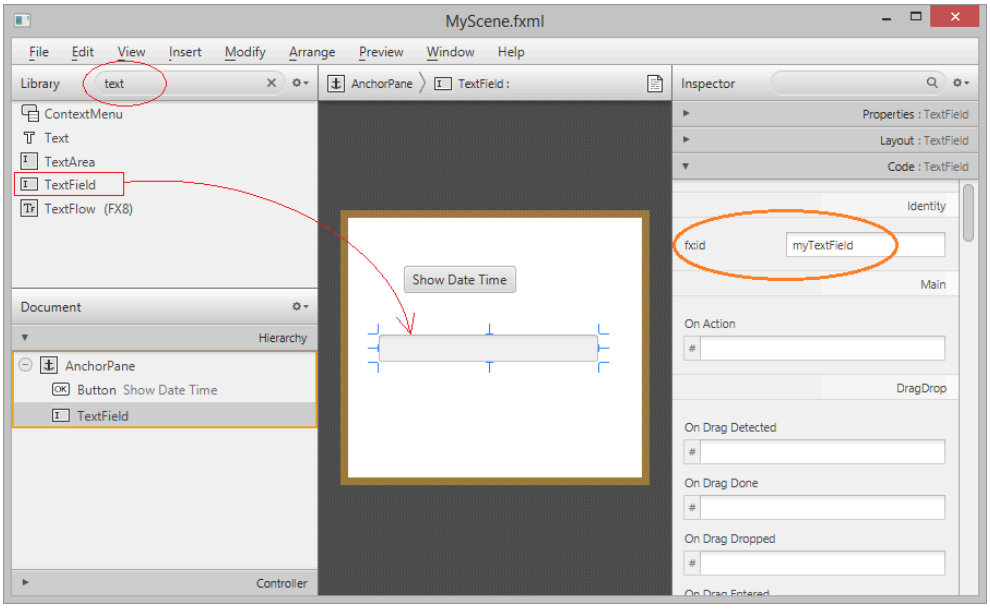
2. Para añadir elementos al UI se pueden arrastrar directamente desde el panel de la izquierda (Library > Controllers) hacia la parte central del Content Panel.

Añade un botón → En el panel de la derecha tienes el control de las propiedades y se podrán ajustar algunas características como el texto del botón

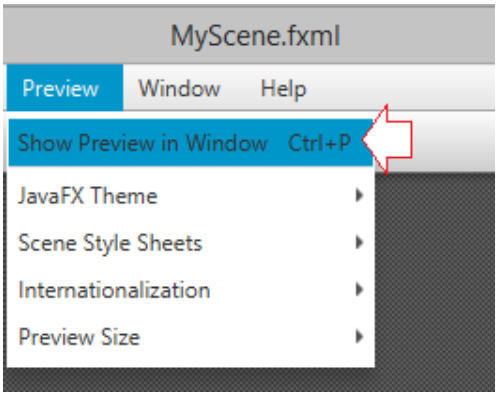


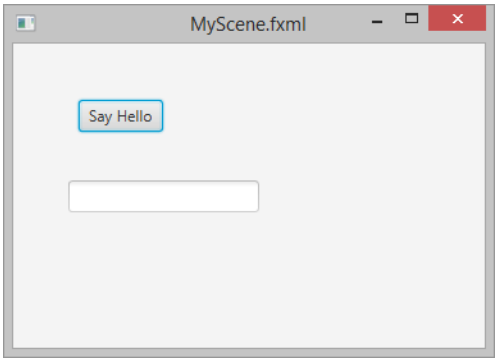
3. Ejecuta la vista previa Preview > Show Preview in Window ó Ctrl +P para ver cómo sería tu interfaz. Si tratas de modificar el tamaño de la ventana, tu botón se comporta de manera aleatoria. Eso puede controlarse en las opciones de layout con las Anchor Pane Constraints que aseguren el número de pixel mínimo o también el padding. ¡Prueba!

4. Añade un campo de texto (o textField)



5. Previsualiza tu interfaz





Para saber más

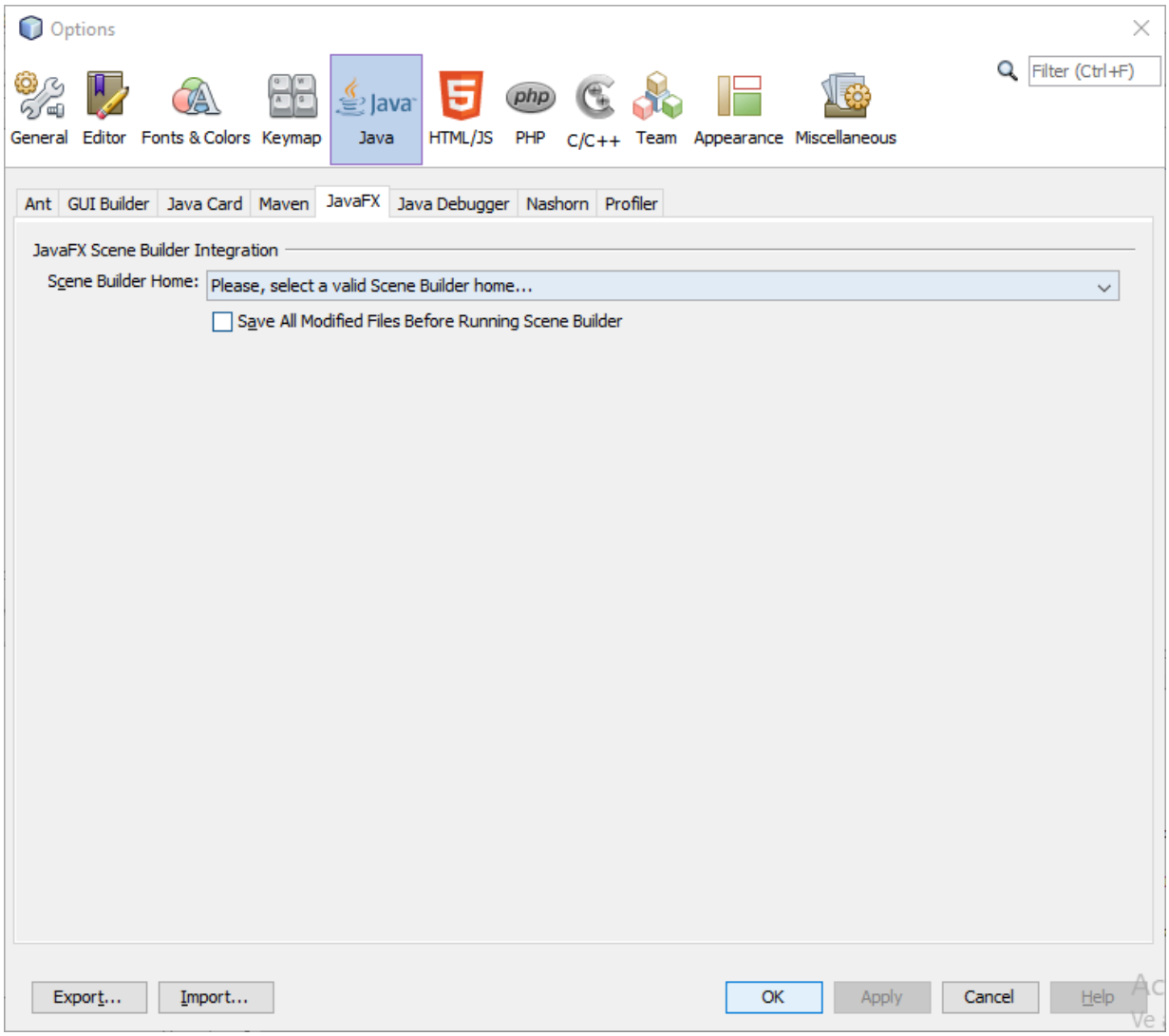
Para coger soltura, descubrir todos los controles y componentes es muy importante practicar con esta herramienta. Existen muchos recursos en internet, aquí os dejamos un pequeño tutorial para principiantes. ¡Sácale partido!

3.2. Integrando JavaFX Scene Builder y NetBeans

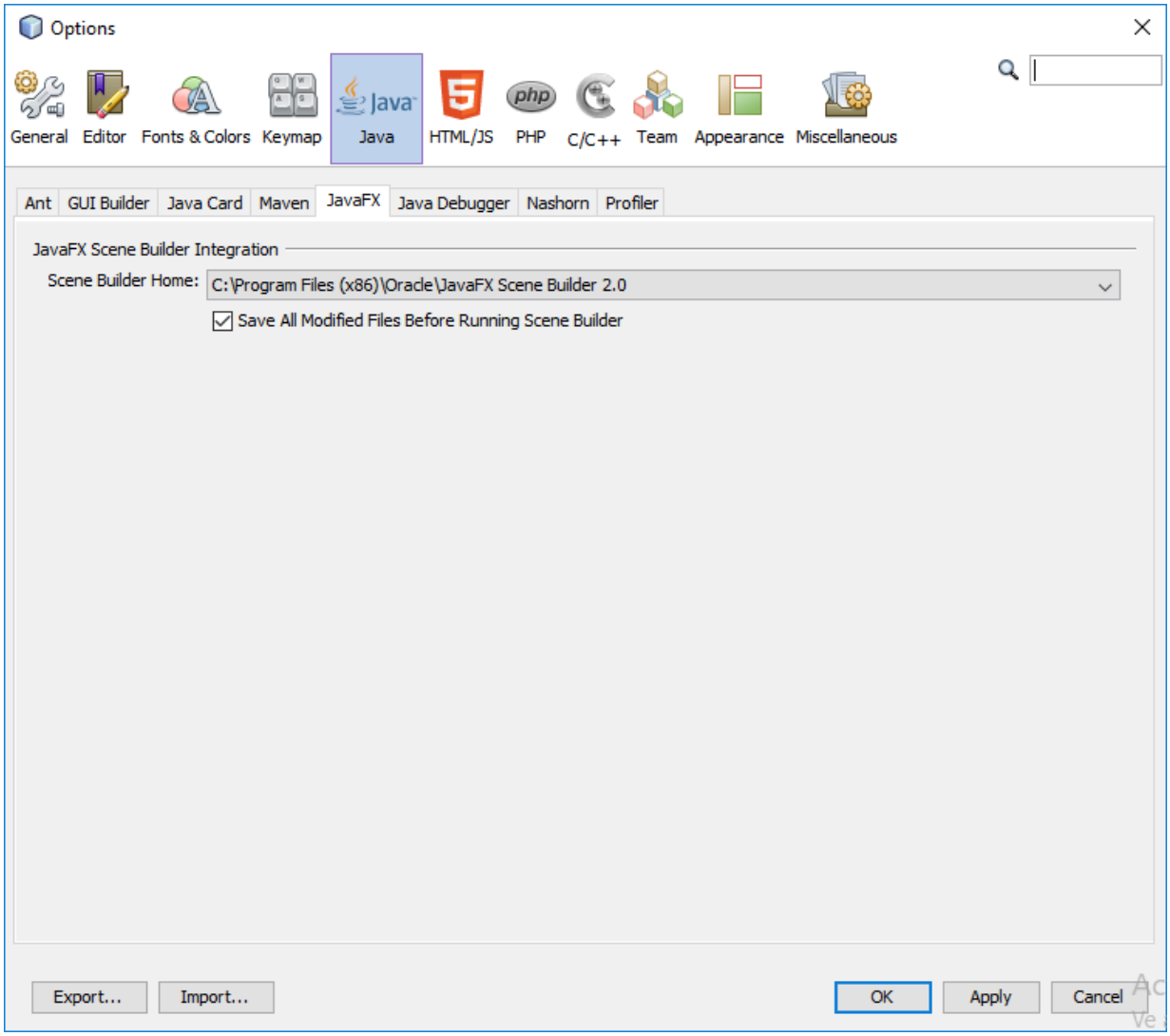
En el subapartado anterior vimos como poder realizar interfaces gráficos de manera sencilla. Poco a poco, os iremos introduciendo más componentes, diferentes layouts y algunos trucos y consejos sobre interfaces gráficas. Ahora lo importante es integrar esta herramienta con nuestros proyectos Java en NetBeans para también poder añadir nuestra lógica de programa y unirla a los interfaces gráficos a través de los eventos.

En el siguiente tutorial veremos cómo integrar JavaFX Scene Builder en NetBeans

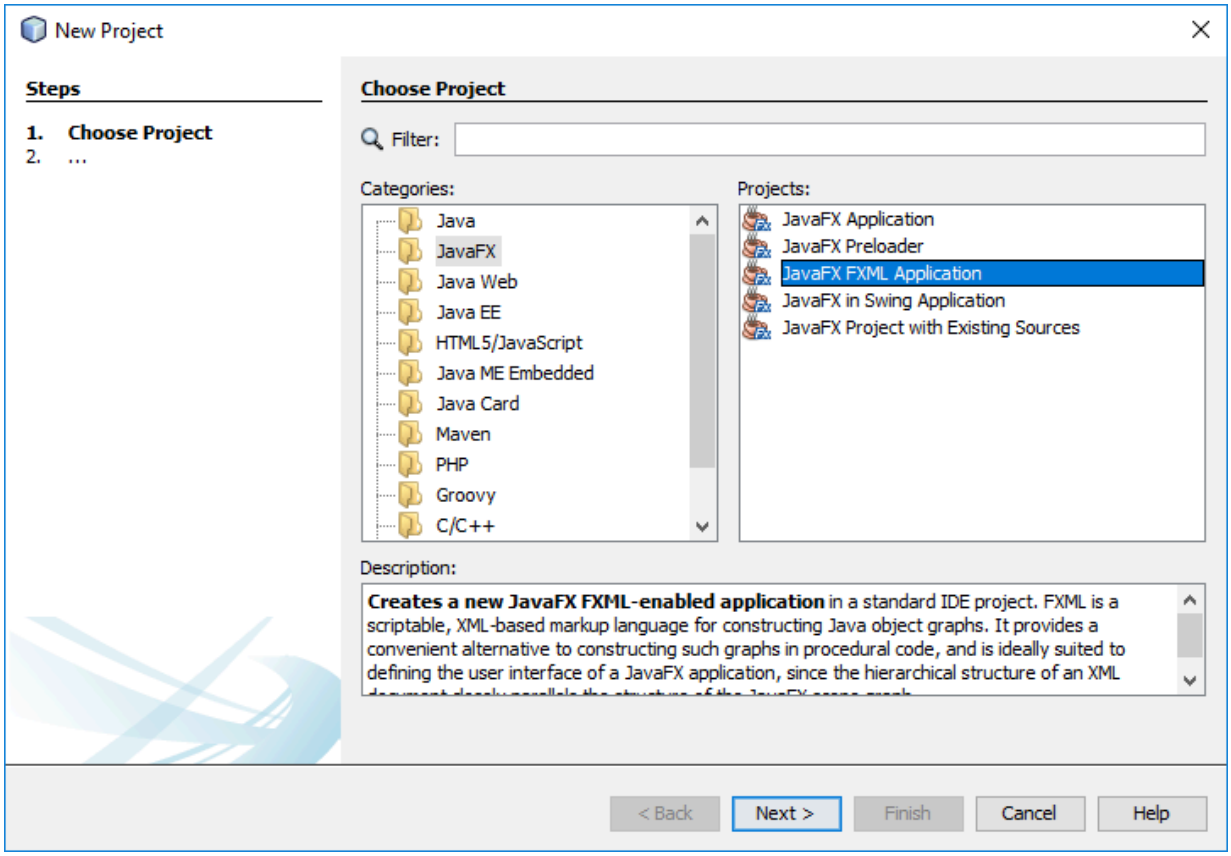
1. Puedes acceder a las opciones de configuración de NetBeans en el menú Tools >Options. Ahí accede a la sección Java y la pestaña JavaFX:



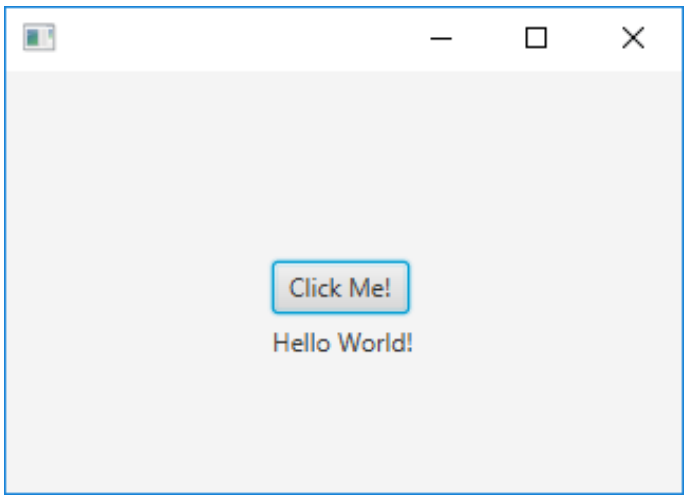
2. Debes de seleccionar la ruta donde instalaste el Scene Builder.



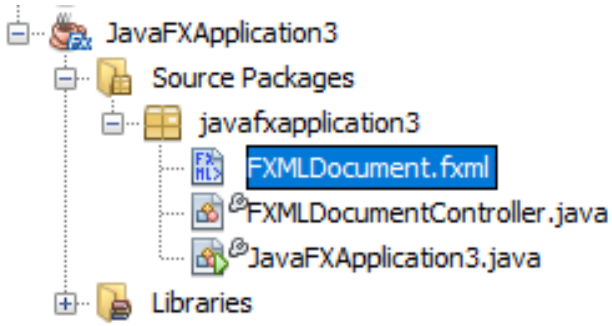
3. A continuación vamos a crear un proyecto JavaFX, pero con la indicación de que vamos a utilizar JavaFX Scene Builder. Para eso, tenemos que escoger la opción JavaFX > JavaFX FXML Application



4. Ponemos el nombre al proyecto que deseemos. Si quieres, antes de analizar el código, ejecútalo. Aparecerá una ventana con un botón, si lo presionas, escribe sobre la propia ventana un mensaje “Hello World”.



5. Vamos a analizar el código y el proyecto. Ahora en la carpeta de fuentes aparecen tres ficheros:



6. La clase java **JavaFXApplication13** es la principal en este proyecto, y será la que inicia la ejecución de la aplicación. Dentro de su código fuente puedes observar que se encarga de **cargar la estructura de la ventana** contenida en el archivo **FXMLDocument.fxml**. En el subapartado siguiente, nos detendremos a explicarlo.

```
public class JavaFXApplication3 extends Application {

    @Override
    public void start(Stage stage) throws Exception {
        Parent root = FXMLLoader.load(getClass().getResource("FXMLDocument.fxml"));

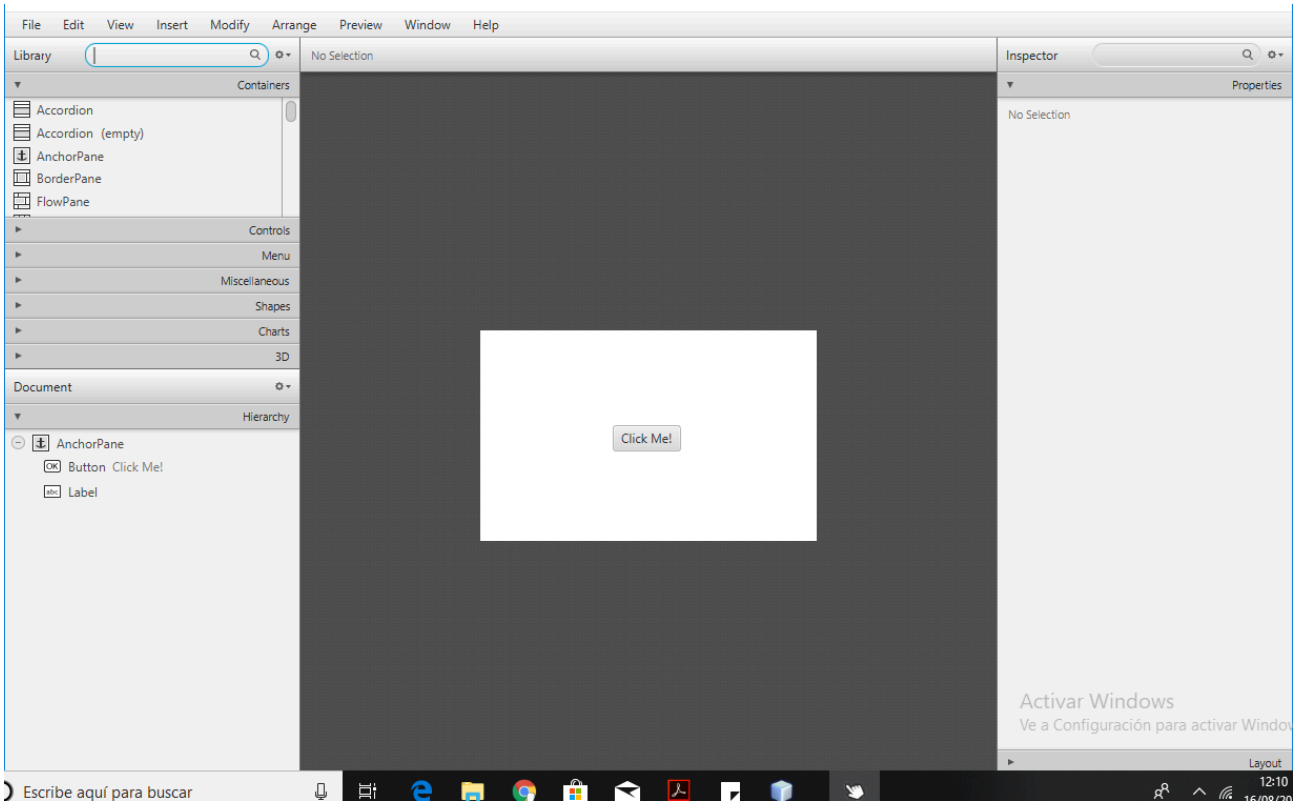
        Scene scene = new Scene(root);

        stage.setScene(scene);
        stage.show();
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        launch(args);
    }

}
```

7. El FXMLDocument.fxml será el interfaz gráfico. Si hacemos doble click sobre él, se nos abrirá el JavaFX Scene Builder con el interfaz gráfico del proyecto, en este caso es sólo un botón. Podremos modificar y guardarlo para trabajar con él.



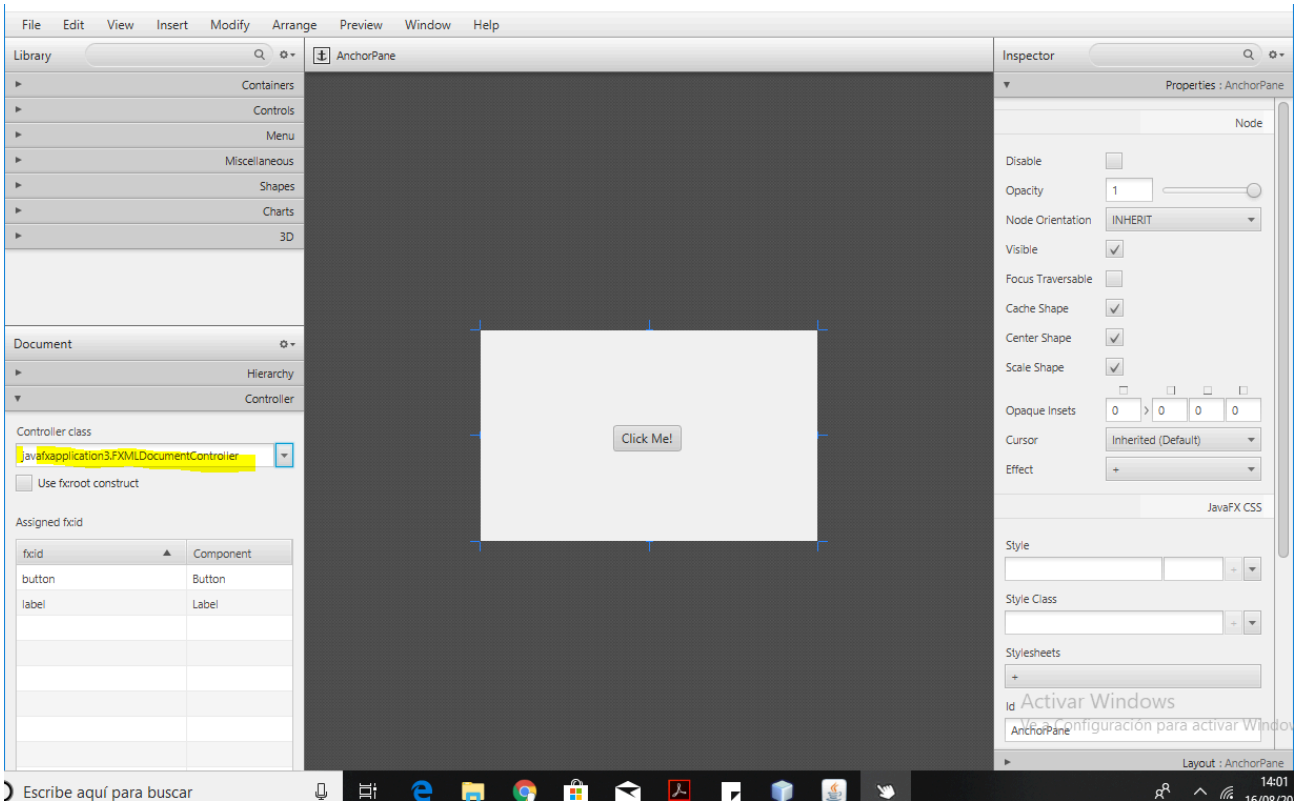
8. Si quieres, puedes ver el código que se va generando sobre este archivo. Al ver su código comprobarás cómo el interfaz gráfico hace referencia al archivo FXMLContorller.java que es la clase controladora del interfaz gráfico, el que va a unir el interfaz gráfico con la lógica de programa.

```
<?xml version="1.0" encoding="UTF-8"?>

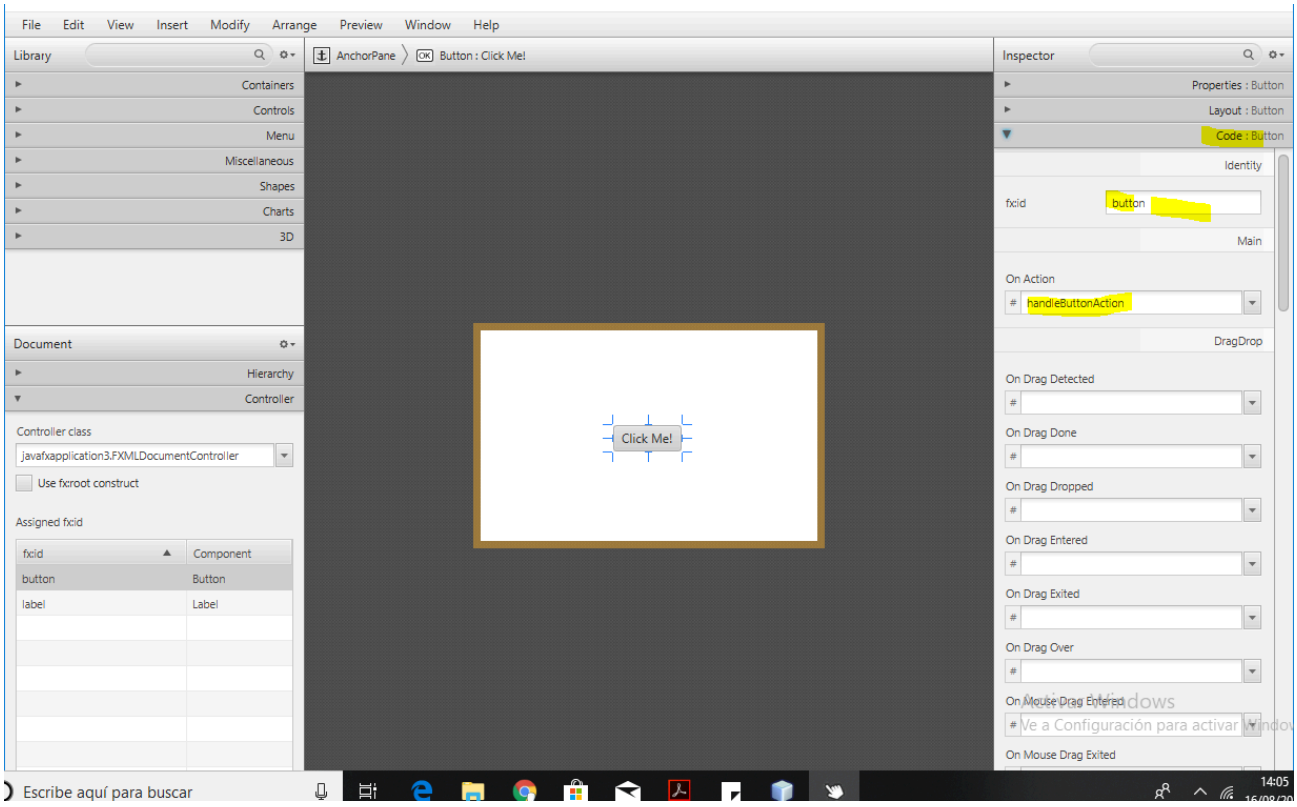
<?import java.lang.*?>
<?import java.util.*?>
<?import javafx.scene.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<AnchorPane id="AnchorPane" prefHeight="200" prefWidth="320" xmlns:fx="http://javafx.com/fxml/1"
            fx:controller="javafxapplication3.FXMLDocumentController">
    <children>
        <Button layoutX="126" layoutY="90" text="Click Me!" onAction="#handleButtonAction" fx:id="button" />
        <Label layoutX="126" layoutY="120" minHeight="16" minWidth="69" fx:id="label" />
    </children>
</AnchorPane>
```

9. Esta vinculación se hace también en el JavaFX Scene Builder., concretamente desde el apartado Controller que puedes encontrar en la parte inferior derecha.



10. Dentro del trozo de **código correspondiente al botón** (Button) en el archivo que se acaba de mostrar, puedes observar que se hace referencia al **método que se ejecutará** cuando el usuario de la aplicación haga clic en él. En concreto se indica que se ejecute el método ***handleButtonAction*** que se encuentra dentro del controlador *FXMLDocumentController*.



11. Si ahora ves el código de la clase controladora, podrás observar cómo cada elemento del interfaz gráfico se referencia con la anotación @FXML (tanto el botón como la etiqueta). Esto es muy importante para poder vincular correctamente el interfaz y la lógica del programa. A medida que vayas creando código, en el interfaz gráfico (a través del Scene Builder) podrás vincular las acciones (eventos) al código correspondiente.

```
public class FXMLDocumentController implements Initializable {

    @FXML
    private Label label;

    @FXML
    private void handleButtonAction(ActionEvent event) {
        System.out.println("You clicked me!");
        label.setText("Hello World!");
    }

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        // TODO
    }

}
```

Repaso

En estos dos videotutoriales puedes ver la manera en que se vincula y crea un proyecto. En la segunda parte, cómo se va incorporando código a la clase controladora y al interfaz. Tutorial JavaFX NetBeans I y Tutorial JavaFX NetBeans II

3.3. Stage y Scene

Una vez realizados estos ejemplos, es conveniente detenernos a explicar el código de la clase principal.

```
public class JavaFXApplication3 extends Application {

    @Override
    public void start(Stage stage) throws Exception {
        Parent root = FXMLLoader.load(getClass().getResource("FXMLDocument.fxml"));

        Scene scene = new Scene(root);

        stage.setScene(scene);
        stage.show();
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        launch(args);
    }

}
```

En este simple código vemos varios puntos importantes, y el primero de ellos es que nuestra clase principal extiende de la clase Application. El método start es el que contiene nuestro primer código en Java FX, pero para entenderlo es necesario aclarar los conceptos de Stage, Scene, Node y Scene Graph.

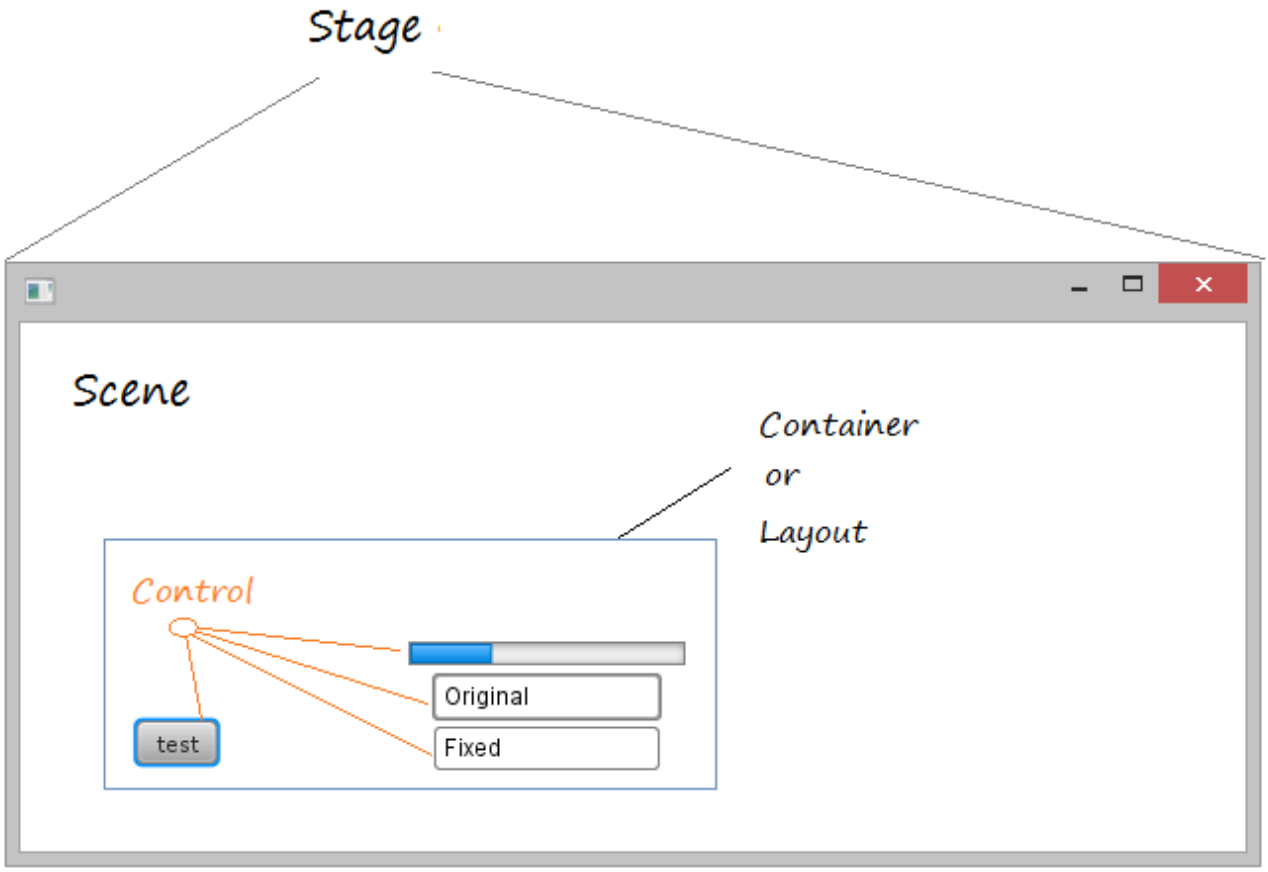
Stage y Scene.

Uno de los símiles que más se aproxima para entender estos conceptos es ver el stage como un escenario, teatro o cine, en el cual, se desarrollan escenas. Dicho de otro modo, stage es un espacio y una escena define que sucede en ese espacio. Visto así, tenemos que un stage es el contenedor de nivel superior, que como mínimo consta de una escena, que a su vez es contenedora de otros elementos. Si nuestro programa es una aplicación de escritorio, el stage será la ventana, con su barra de título, y botones de maximizar, minimizar o cerrar.

Nodos y Scene Graph.

Los nodos son elementos individuales que forman una escena, como por ejemplo botones, cuadros de texto, layouts etc. Si seguimos con el símil de un teatro, los nodos serían los actores y el atrezo. Sin embargo, en Java FX los nodos pueden tener nodos hijos, que a su vez tienen otros nodos hijos, como si de las ramas de un árbol se tratase. El conjunto de todos los nodos que forman una escena es lo que llamamos Scene Graph o grafo de la escena.

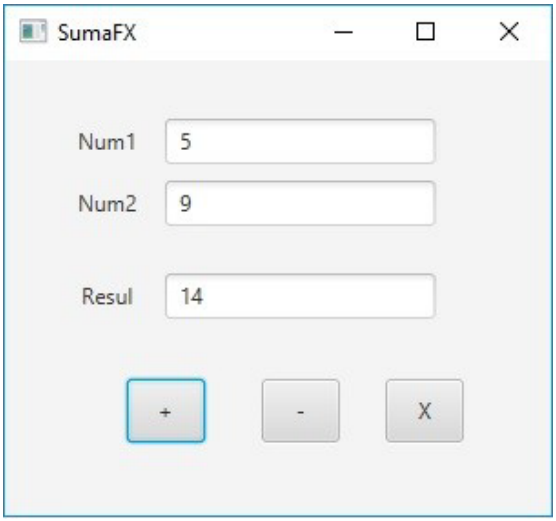
En Java FX existe un nodo especial llamado root node o nodo raíz/padre del cual extienden todos los demás nodos, y es el único que no tiene un nodo padre, por que todos los demás nodos si lo tienen y son descendientes ya sea directa o indirectamente del root node. Todos los nodos extienden de la clase Node, y existen otras clases que extienden directa o indirectamente de Node, como Parent, de la cual ya tenemos un objeto en nuestra plantilla por defecto, y por nombrar algunas más tenemos Group, Region o Control.



Una vez entendidos los conceptos, en nuestra clase principal HolaJavaFX.java como puedes observar lo primero que hacemos es cargar todos los nodos hijos, en el root node, después creamos una escena y añadimos la escena a nuestro stage, por último mostramos con el método .show() nuestro stage.

Ejercicio Resuelto

Seguir los pasos de un tutorial tan guiado es fácil, pero a veces no entendemos muy bien qué es lo que estamos haciendo y estamos inseguros ante la idea de abordarlo por nosotros mismos con garantías. Para reforzar los aspectos básicos que hemos visto en el tutorial, enlazar el interfaz gráfico con los manejadores, seguir practicando con Builder Scene y conocer otros componentes se propone realizar una pequeña aplicación de calculadora muy sencilla. El interfaz gráfico ha de realizarse con JavaFX debe ser algo parecido a lo que se muestra en la imagen. La funcionalidad: La aplicación espera a que se introduzca valores y se presione alguno de los botones.(puedes incorporar el código todos los chequeos y comprobaciones que consideres).



Ejercicio Resuelto

Para saber más

Esta manera de separar la lógica del programa de los interfaces gráficos y el modelo de datos recibe el nombre del **Patrón Modelo Vista Controlador o MVC**. Es una arquitectura del software de cómo ordenarlo para modularizarlo e independizarlo. Es una muy buena práctica de programación. Puedes leer más en el artículo sobre el MVC de Wikipedia

4. Layouts y Componentes

Como hemos visto, el código de los proyectos JavaFX se organiza a través de diferentes clases como Stage, Scene, donde se pueden agrupar los diferentes contenedores o layouts, donde se podrán distribuir los diferentes componentes o controles (que a su vez dispararán los eventos o acciones que se programarán en las clases controladoras).

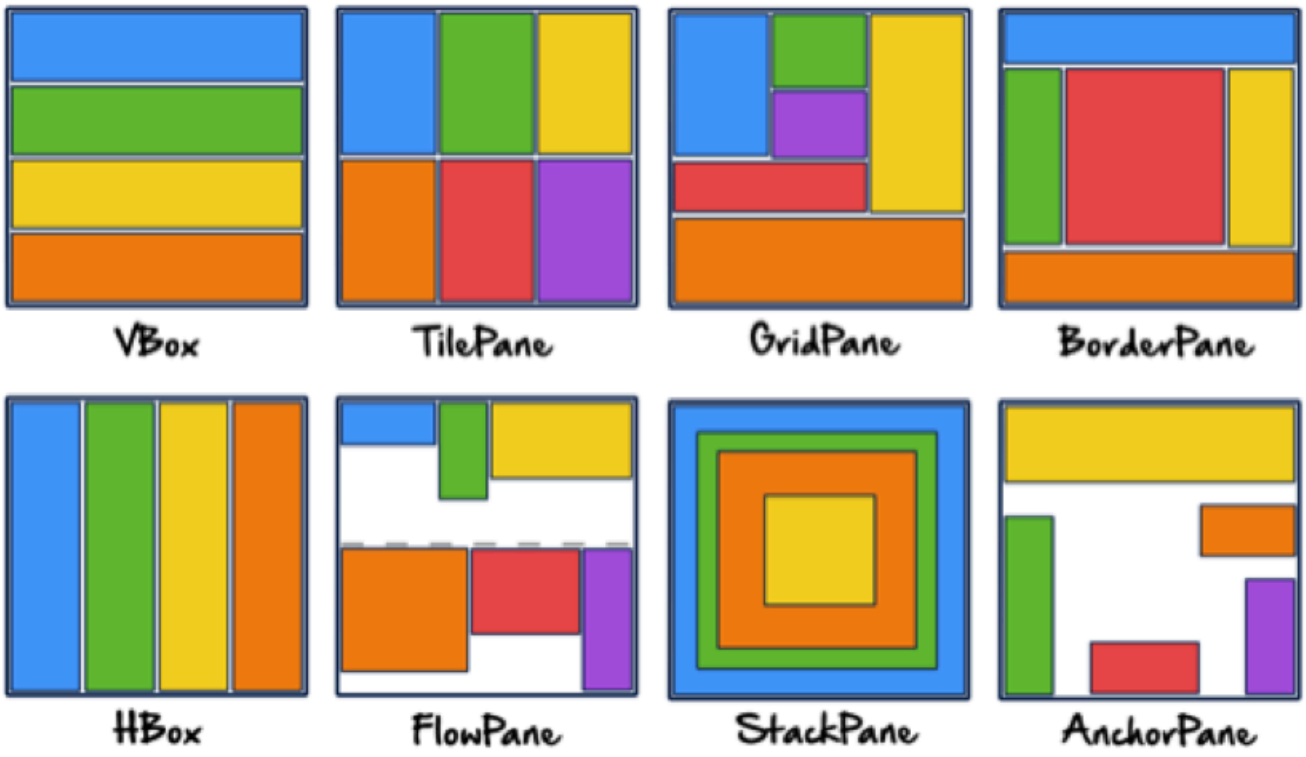
En este gráfico se explica de manera visual muy explícitamente.

En diseño y programación se conoce como **Layout** al esquema de distribución de los elementos dentro un interfaz. En JavaFX las clases contenedoras de layout se denominan **Pane**.

JavaFX proporciona varios **layout panes** para gestionar los diferentes diseños (filas, columnas, etc). Lo importante para escoger un layout pane es saber cómo se va a comportar cuando se redimensiona una ventana, pues el layout pane reposicionará y redimensionará los elementos que contiene en función de cómo configuremos sus propiedades.

Los layout panes de JavaFX son los siguientes:

- **FlowPane** – Dispone sus elementos consecutivamente dentro de los límites del panel. Pueden comportarse horizontal o verticalmente.
- **HBox** – dispone sus elementos horizontalmente en una única fila.
- **VBox** – dispone sus elementos verticalmente en una única columna.
- **AnchorPane** – permite distribuir los elementos en cuatro regiones: top, bottom, left side, or center.
- **BorderPane** – permite distribuir los elementos en cinco regiones: top, bottom, right, left, or center region.
- **StackPane** – ubica los elementos de atrás hacia delante en una única pila.
- **TiledPane** – permite distribuir los elementos en celdas o mosaicos del mismo
- **GridPane** – permite distribuir los elementos en una rejilla de filas y columnas.
- **SplitPane** – permite dividir la pantalla en dos áreas, bien verticalmente o bien horizontalmente.



Se pueden crear layout más complejos empleando diferentes contenedores.

Debes conocer

En este amplio tutorial sobre JavaFX puedes encontrar ejemplos y trucos para generar un ejemplo con cada uno de estos layouts panes. Aquí te dejamos el primero, sobre FlowPane. En el índice a la izquierda, podrás encontrar

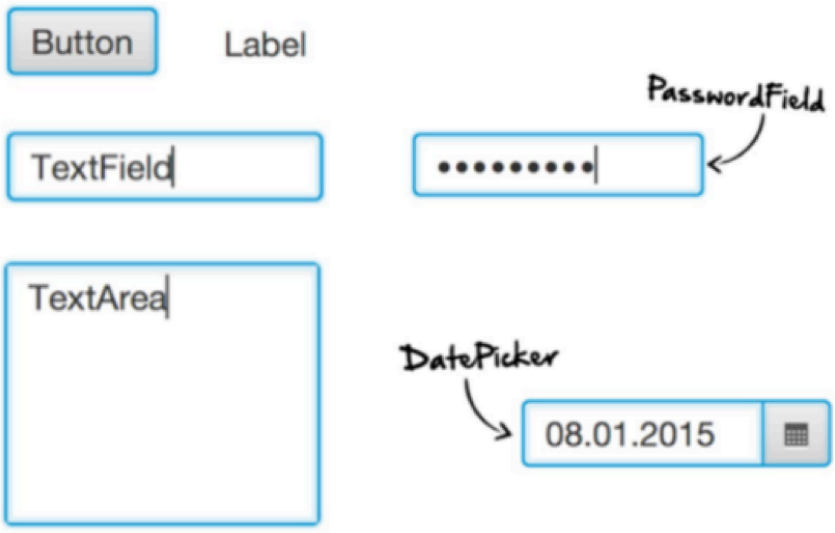
el resto. Te recomendamos practicar con ellos.

4.1. Componentes

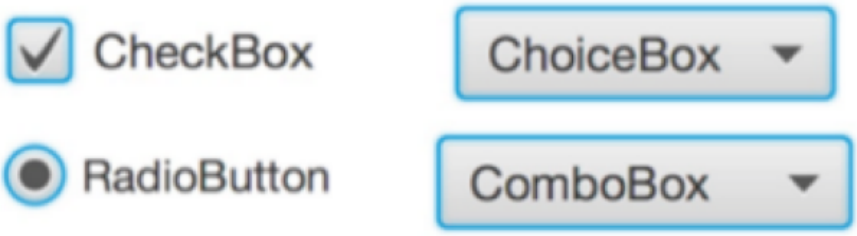
Los componentes son los diferentes elementos y controles que se van a poder incorporar en el layout y que conformarán nuestros interfaces gráficos. Serán clases Java que se tendrán que instanciar como un objeto (recuerda que el Scene Builder generará el código). En JavaFX existen muchos y bien variados: etiquetas (label), botones (button), radiobutton, checkbox, campos de texto (textFiled), listas (listview), tablas (TableView), etc. . Recuerda cuando ejecutasteis la demo Mondena.jar y pudiste ver muchos de ellos.

Estos son algunos de los más relevantes, lo llamamos por su clase, que mostramos gráficamente. Después se facilitan enlaces a la documentación oficial y otros recursos donde hay desarrollados ejemplos de cada uno de ellos.

- **Button**. Clase Button para incorporar un botón.
- **Label**. Etiqueta de texto que no puede ser modificada por el usuario.
- **TextField**. Campo de texto para recoger información del usuario. Suele ser una línea. Fundamental en los formularios.
- **TextArea**. Campo de texto mayor que el TextField, con la misma finalidad.
- **PasswordField**. Similar al TextField pero el texto introducido se visualiza con puntos para ocultarlo. Es ideal para recoger contraseñas.
- **DatePicker**. Sirve para solicitar una fecha. Si se presiona el icono de la derecha se abrirá un calendario para seleccionar la fecha.
- **ToggleButton**. Tiene el aspecto de un botón pero, a diferencia de Button, no se utiliza para disparar un evento, si no para saber si se está pulsado o no. Realmente se comporta como un interruptor de dos posiciones.



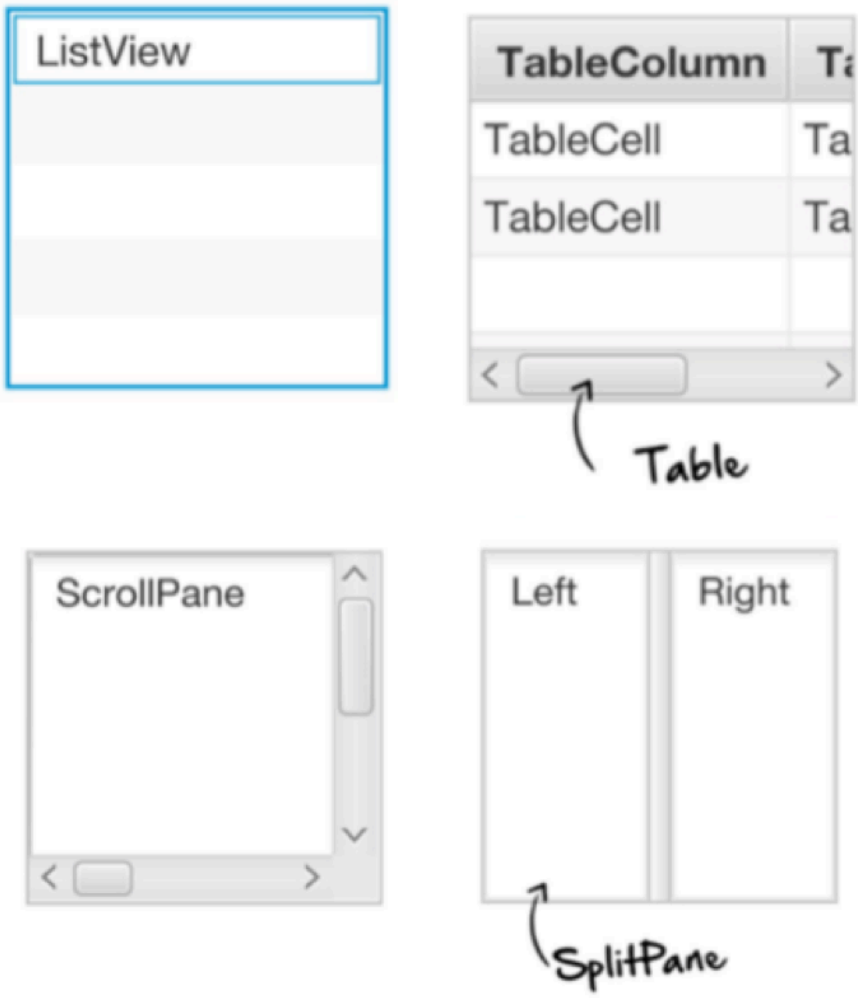
- **CheckBox**. es un elemento de interacción con el usuario, que permite a éste hacer selecciones múltiples de un conjunto de opciones.
- **RadioButton**. Es un elemento de interacción que permite hacer una selección única entre varias.
- **ChoiceBox**. Es un menú de selección para que el usuario escoja una de las opciones que se le despliegan de dicho menú.
- **ComboBox**. Es muy similar al choicebox, con la salvedad que las opciones a escoger son editables, pudiéndose introducir o editar las opciones a seleccionar.



Para mostrar datos tenemos diferentes opciones de listas, tablas, etc

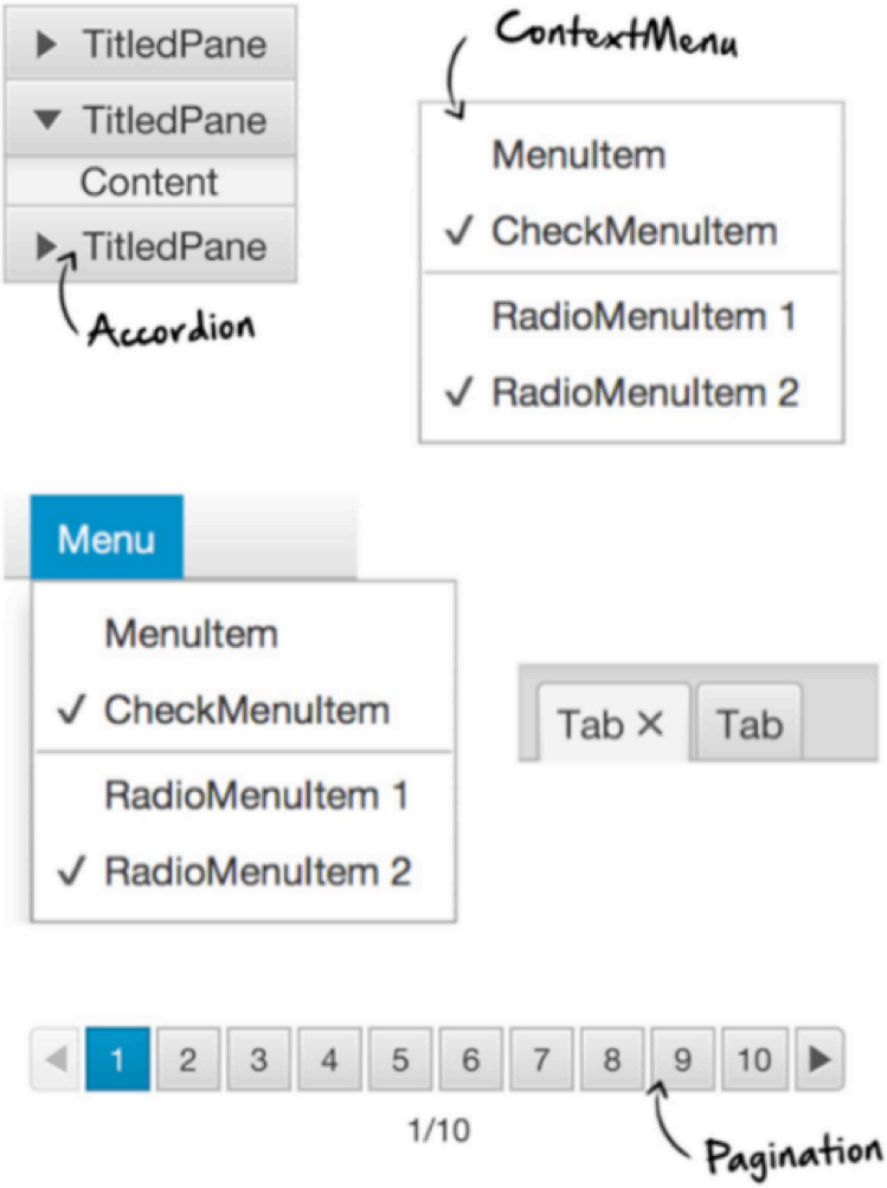
- **ListView.** Servirá para mostrar datos agrupados en un modo lista.

- **TreeView.** Muy similar a ListView pero con la opción de que los elementos de la lista se agrupen.
- **Table.** Servirá para mostrar datos en forma de tabla con columnas y filas programables.
- **ScrollPane**. Permite mostrar información en espacio más reducido habilitando el scroll horizontal y vertical para completar su visualización.
- **SplitPane.** Permite dividir la información en dos partes a través de un separador.



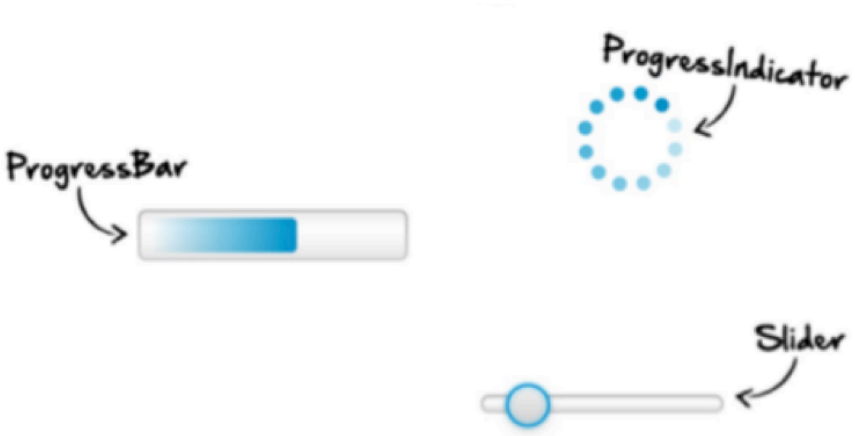
Tenemos diferentes opciones para diseñar menús y navegar entre diferentes ventanas.

- **Menu**. El Menu es la forma en el que se desplegarán las opciones MenuItem a modo de barra de menú arriba de la ventana.
- **ContextMenu**. Cuando no tenemos espacio en nuestro interfaz, tenemos la opción de que un evento “dispare” un menú contextual.
- **Accordion.** Muy similar al Menu pero se agruparán los submenus.
- **Pagination.** Se utiliza cuando la información a mostrar se divide en páginas que se pueden seleccionar.
- **Tab.** Sirve para hacer un menú en forma de pestañas.



Otros componentes:

- ProgressBar.
- ProgressIndicator
- Slider



Debes conocer

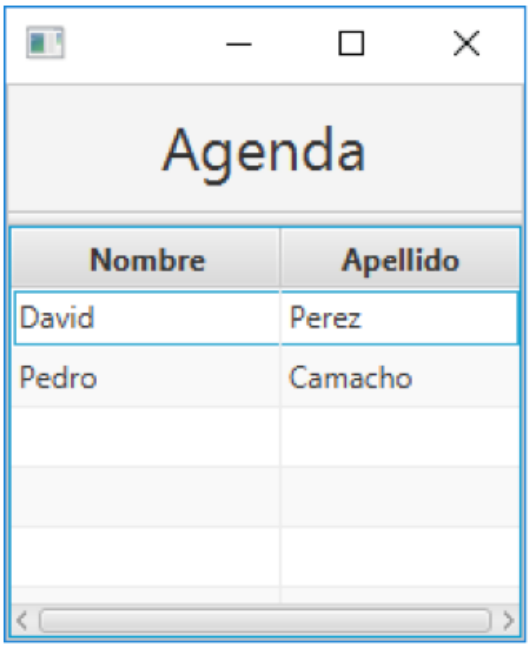
En el siguiente tutorial tienes ejemplos de cada uno de los controles de manera detallada utilizando Scene Builder:
Tutorial Controles JavaFX
También puedes encontrar ejemplos y guía en la Página Oficial de Oracle sobre los Controles Java FX

Ejemplo I

En el siguiente videotutorial podrás ver cómo realizar un ejemplo de **Aplicación GUI con JavaFX** en el que utilizar controles sencillos como campos de texto, botones, choicebox, etc.

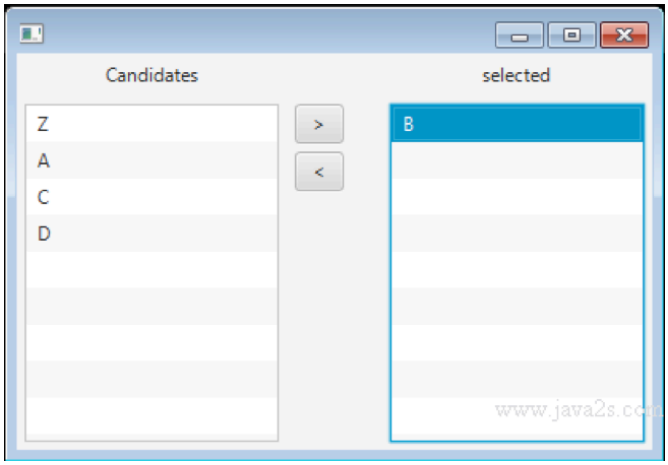
Ejemplo II

En este ejemplo vas a poder ver cómo incluir una tabla con datos en tu interfaz gráfico. El ejemplo no tiene ninguna operativa, sólo visualiza varios elementos metidos por defecto en el código en la tabla. Lo importante del ejemplo es aprender a utilizar la clase ObservableList que facilita el trabajo con las tablas y la creación de una clase que para ordenar los datos a visualizar. Descargate y analiza TablaFX.Zip.



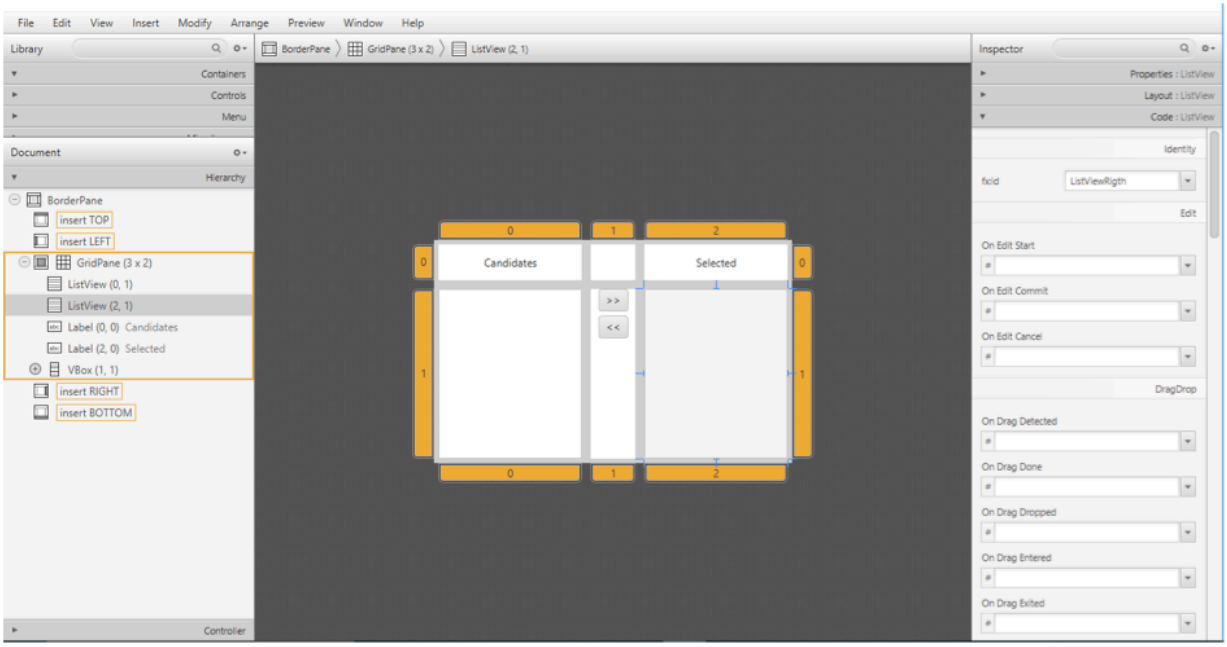
Ejercicio Resuelto

Vamos a realizar una pequeña aplicación que muestre una serie de elementos en un ListView y mediante la acción de los dos botones podamos editar la lista de candidatos y seleccionados



Pistas:

- Crea un nuevo Proyecto JavaFX -> UIScene.FXML (BorderPane)
- En el SceneBuilder crea un BorderPane (dimensiones aprox 400 x 300)
- Dentro de la parte central del BorderPane ubica un GridPane de 3 columnas x 2 filas.
- Define una separación entre columnas de 10px (Hgap, Vgap)
- Establece un padding de 5px
- Ahora hay que definir las propiedades de cada columna:
    - Las dimensiones de las columnas 1 y 3 serán 150 de alto y 150 ancho
    - La dimensión de la columna 2 = 50px de ancho y 150 alto.
    - Hay que definir la prioridad Hgrow como ALWAYS
- Añade los ListView a las columnas 0,1 y 2,1
- Añade los botones a la columna 1,1
- Añade las etiquetas “Candidates” y “selected” en las columnas 0,0 y 0,2



- Tienes que nombrar en cada elemento su fx:id para referenciarlo en la clase controladora. Echa un vistazo a los ejemplos de listview.
- La clase controladora deberás incluir todos los controles con su correspondiente @FXML
- Utiliza el método initialize para cargar los valores -> ObservableList y utiliza el método setItems para “cargar” los elementos en el listview
- Crea el método para que se ejecute cuando se presione el botón >> Chequea que si no se ha seleccionado nada, se salga. Si se ha seleccionado algo, tiene que añadirlo en el ListView de la derecha
- Crea el método para que se ejecute cuando se presione el botón >> tiene que estar algún elemento seleccionado y si es así, lo borrará
- Como posibles mejoras haz que los elementos no se repitan en selected.

Ejercicio Resuelto.

Para saber más

Construir interfaces gráficos es simple, ya lo habéis visto, pero hacer buenos interfaces gráficos que sean del gusto del usuario que sean usables y les proporcionen una buena experiencia de usuario (UX), no es tan sencillo y además, tiene mucha teoría detrás y aspectos que aprender. En el segundo curso tanto de DAM como de DAW hay un módulo relacionado con los interfaces donde aprenderéis mucho sobre esto. De todas formas, aquí os dejamos un link a un interesante artículo donde se dan recomendaciones para diseñar buenos formularios.

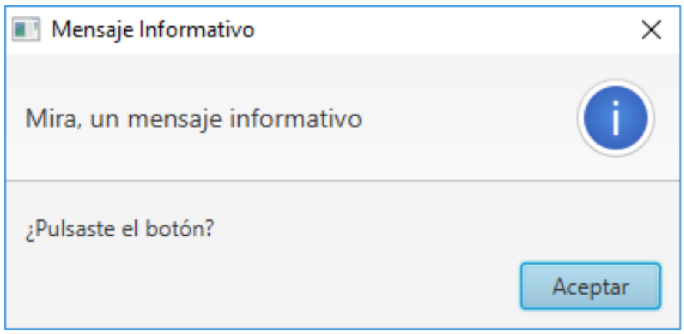
4.2. Ventanas de Alerta: Dialog Box

Java FX incorpora los **dialog box**, esas ventanas de aviso, error, informativos de cualquier aplicación con un entorno gráfico de usuario que necesitaremos para completar la interacción con nuestro usuario.

Existen los diálogos estándares como: información, advertencia, error, excepción, confirmación con acciones customizadas, etc.

Estos dialog box se lanzarán utilizando la clase **Alert**. Puedes ver toda la información de esta clase en la documentación Oficial de Java:

Ejemplo



Esto se consigue con el siguiente código que puedes incorporar en cualquier parte del código.

```
Alert alert = new Alert(AlertType.INFORMATION);
alert.setTitle("Mensaje Informativo");
alert.setHeaderText("Mira, un mensaje informativo");
alert.setContentText("¿Pulsaste el botón?");

alert.showAndWait();
```

Debes conocer

En este post, se realiza un análisis exhaustivo de todos los tipos de Dialog Box y mensajes de alerta que vas a poder utilizar, así como el código que necesitarías para ello. Ejemplos JavaFX Dialog Box

5. Eventos

Caso práctico

Ana sabe que entender cómo funciona la programación por eventos es algo relativamente fácil, el problema está en utilizar correctamente los eventos más adecuados en cada
momento. Ana tiene muy claro que el evento se asocia a un botón cuando se pulsa, pero José Javier la pone en duda, cuando la llama por teléfono para preguntarle unas dudas y le dice, que él cree, que el evento se produce cuando el botón se suelta. Además, le recuerda que en clase dijeron que al ser enfocado con el ratón, o al accionar una combinación de teclas asociadas, también se pueden producir eventos. Tras hablarlo, piensan que realmente no es tan complicado, porque se repiten muchos eventos y si nos paramos a pensarlo, todos ellos son predecibles y bastante lógicos.

5.1. ¿Qué son los eventos?

A estas alturas de la unidad, creemos que ya sabes qué son los eventos. De hecho, hemos manejado alguno de ellos en los ejemplos y mini ejercicios que hemos desarrollado hasta el momento. Pero dada su relevancia, hemos pensado que es necesario deternos un poco y hablaros un poco más de ellos.

¿Qué es un **evento**?

Es todo hecho que ocurre mientras se ejecuta la aplicación. Normalmente, llamamos evento a cualquier interacción que realiza el usuario con la aplicación, como puede ser:

- pulsar un botón con el ratón;
- hacer doble clic;
- pulsar y arrastrar;
- pulsar una combinación de teclas en el teclado;
- pasar el ratón por encima de un componente;
- salir el puntero de ratón de un componente;
- abrir una ventana;
- etc.

¿Qué es la **programación guiada por eventos**?

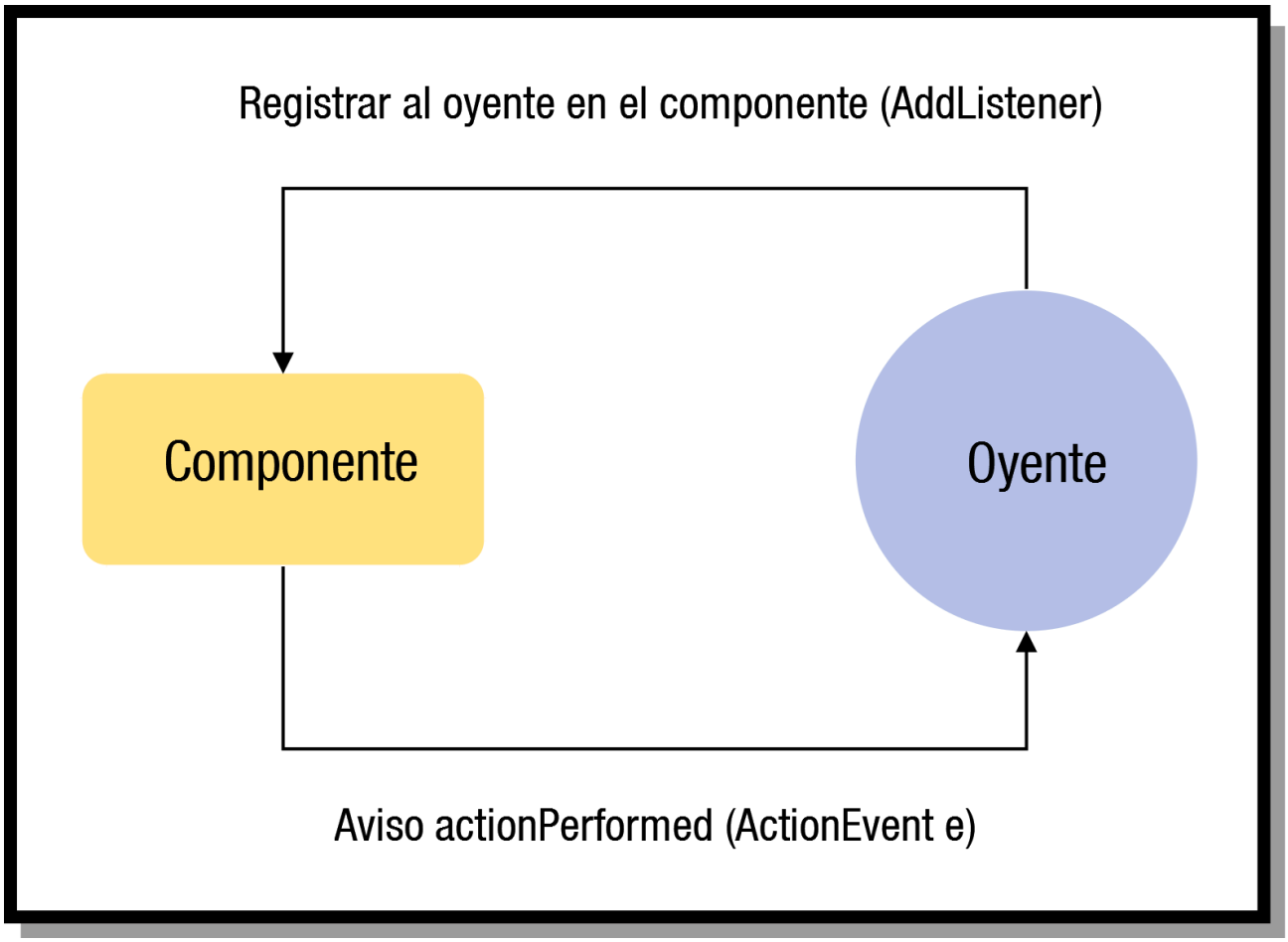
Imagina la ventana de cualquier aplicación, por ejemplo la de un procesador de textos. En esa ventana aparecen multitud de elementos gráficos interactivos, de forma que no es posible que el programador haya previsto todas las posibles entradas que se pueden producir por parte del usuario en cada momento.

Con el control de flujo de programa de la **programación imperativa**, el programador tendría que estar continuamente leyendo las entradas (de teclado, o ratón, etc) y comprobar para cada entrada o interacción producida por el usuario, de cuál se trata de entre todas las posibles, usando estructuras de flujo condicional (**if-then-else, switch**) para ejecutar el código conveniente en cada caso. Si piensas que para cada opción del menú, para cada botón o etiqueta, para cada lista desplegable, y por tanto para cada componente de la ventana, incluyendo la propia ventana, habría que comprobar todos y cada uno de los eventos posibles, nos damos cuenta de que las posibilidades son casi infinitas, y desde luego impredecibles. Por tanto, de ese modo es imposible solucionar el problema.

Para abordar el problema de tratar correctamente las interacciones del usuario con la interfaz gráfica de la aplicación hay que cambiar de estrategia, y la **programación guiada por eventos es una buena solución**, veamos cómo funciona el modelo de gestión de eventos.

5.2. Modelo de gestión de eventos

¿Qué sistema operativo utilizas? ¿Posee un entorno gráfico? Hoy en día, la mayoría de sistemas operativos utilizan interfaces gráficas de usuario. Este tipo de sistemas operativos están **continuamente monitorizando el entorno para capturar y tratar los eventos** que se producen.



El sistema operativo informa de estos eventos a los programas que se están ejecutando y entonces cada programa decide, según lo que se haya programado, qué hace para dar respuesta a esos eventos.

Cada vez que el usuario realiza una determinada acción sobre una aplicación que estamos programando en Java, un clic sobre el ratón, presionar una tecla, etc., se produce un evento que el sistema operativo transmite a Java.

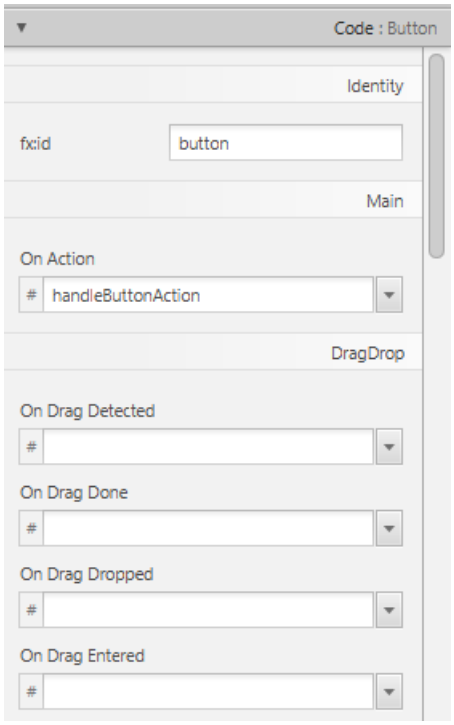
Java crea un objeto de una determinada clase de evento, y este evento se transmite a un determinado método para que lo gestione.

El **modelo de eventos de Java está basado en delegación**, es decir, la responsabilidad de gestionar un evento que ocurre en un objeto fuente la tiene otro objeto oyente.

Las **fuentes de eventos** (event sources) son objetos que detectan eventos y notifican a los receptores que se han producido dichos eventos. Ejemplos de fuentes:

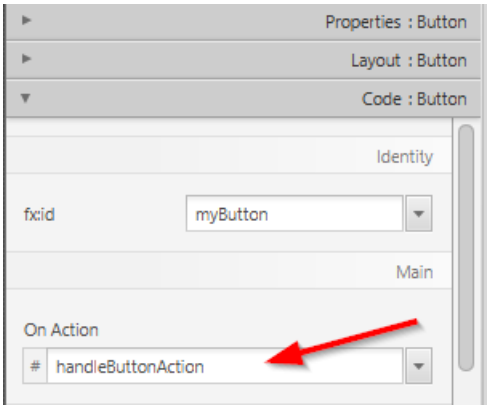
- Botón sobre el que se pulsa o pincha con el ratón.
- Campo de texto que pierde el foco.
- Campo de texto sobre el que se presiona una tecla.
- Ventana que se cierra.
- Etc.

En el apartado anterior de creación de interfaces con ayuda del Scene Builder, vimos lo fácil que es realizar este tipo de programación, ya que el Scene Builder y el propio NetBeans hacen muchas cosas como generar código automáticamente por nosotros. Además, cada componente, en el apartado *Code*, mostraba todos los eventos a los que se le podía añadir un disparador o método de la clase controladora de la ventana.



Recuerda que en la clase controladora, debes de añadir la anotación @FXML para que sea visible en el SceneBuilder

```
@FXML
private void handleButtonAction(ActionEvent event) {
    // Button was clicked, do something...
    outputTextArea.appendText("Button Action\n");
}
```



Pero también podríamos hacerlo nosotros todo, si no tuviéramos un IDE como NetBeans, o porque simplemente nos apeteciera hacerlo todo desde código, sin usar asistentes ni diseñadores gráficos.

En el API JAvaFX la clase base que representa los eventos es **javafx.event.Event** todas las subclases de la misma también representan eventos, algunas de estas clases son:

- **MouseEvent**: Representa los eventos producidos por la interacción del ratón, por ejemplo, hacer clic, mover el cursor, girar la rueda, etc.
- **KeyEvent**: Esta clase es para los eventos producidos por el teclado, puede ser, presionar o liberar un tecla.
- **WindowEvent**: Aquí tenemos los eventos producidos por la ventana, por ejemplo, el mostrar u ocultar la ventana.
- En la última versión del API JavaFX 2.2 se han añadido otro tipo de eventos relacionado con pantallas táctiles y la programación móvil, como los **TouchEvents, DragEvents, ScrollEvents**, etc.

Debes conocer

En la documentación oficial de JavaFX puedes encontrar todas las clases de eventos que se pueden manejar.fdgfdgdsvvsv

Para controlar un evento en JavaFX disponemos de los manejadores de eventos y los filtros de eventos, cada uno de estos controladores poseen estas propiedades:

- **Target**: El nodo en donde se produjo el evento, un botón, la ventana, etc..
- **Source**: Indica la fuente que genera el evento, el teclado, el ratón, etc..
- **Type**: Se refiere el tipo de evento producido, presionar un botón del mouse, presionar una tecla, mover el mouse, etc..

Para responder a un evento requerimos un objeto que implemente la interface EventHandler

Para agregar estos manejadores de eventos usaremos los métodos addEventHandler() o addEventFilter(), debemos indicar como parámetros, el tipo de evento que deseamos manejar, y el manejador de eventos, del mismo modo removemos o quitamos un manejador usando las funciones, removeEventFilter() o removeEventHandler().

Para saber más

Si quieres ampliar tus conocimientos para gestionar los eventos manualmente, puedes acceder a este post donde se explica con ejemplos de código y puedes descargar el código.

También os facilitamos el tutorial completo proporcionado por Oracle para la gestión manual de eventos.

6. Aplicar estilos con CSS

Esperamos que conozcas qué son las **hojas de estilo CSS**. Si estás cursando el ciclo de **Desarrollo de Aplicaciones Web** o de **Desarrollo de Aplicaciones Multiplataforma**, estamos convencidos que lo has visto en el módulo de **Lenguaje de Marcas y Sistemas de Gestión de la Información**. No son los objetivos de esta unidad aprender CSS, pero sí, si lo conoces, aplicar algunas técnicas para cambiar el estilo de tus vistas que se aplican allí.

Debes conocer

CSS es un lenguaje que describe el estilo de un documento HTML. CSS describe cómo debe ser mostrados los elementos HTML. Te dejamos acceso a un tutorial sobre CSS del W3Cschool en caso de que no lo conozcas.

Como os decíamos, JavaFX nos permite incluir hojas de estilo CSS para separar el contenido de la interfaz gráfica de su aspecto, justo como se hace en el diseño Web.

6.1. Estilo por defecto en JavaFX; Modena.css

Los estilos por defecto de JavaFX 8 se encuentran en un archivo denominado modena.css. Este archivo CSS se encuentra dentro del archivo jar jfxrt.jar que se encuentra en tu directorio de instalación de Java, en la ruta /jdk1.8.x/jre/lib/ext/jfxrt.jar.

Puedes descomprimir jfxrt.jar o abrirlo como si fuera un zip. Encontrarás el archivo modena.css en la ruta com/sun/javafx/scene/control/skin/modena/

También puedes abrirlo desde aquí modena.css.

Este estilo se aplica siempre a una aplicación JavaFX. Añadiendo un estilo personal podemos reescribir los estilos por defecto definidos en modena.css. Os dejamos una captura de ese archivo para ver que los elementos no son los del HTML si no los de JavaFX.

```
/*******************************************************************************
 *                                                                             *
 * RadioButton                                                                 *
 *                                                                             *
 ******************************************************************************/

.radio-button {
    -fx-label-padding: 0.0em 0.0em 0.0em 0.416667em; /* 0 0 0 5 */
    -fx-text-fill: -fx-text-background-color;
}
.radio-button > .radio,
.radio-button:focused > .radio  {
   -fx-background-radius: 1.0em; /* large value to make sure this remains circular */
   -fx-padding: 0.333333em; /* 4 -- padding from outside edge to the inner black dot */
}
.radio-button > .radio > .dot {
   -fx-background-color: transparent;
   -fx-background-radius: 1.0em; /* large value to make sure this remains circular */
   -fx-padding: 0.333333em; /* 4 -- radius of the inner black dot when selected */
}

/*******************************************************************************
 *                                                                             *
 * CheckBox                                                                    *
 *                                                                             *
 ******************************************************************************/

.check-box {
    -fx-label-padding: 0.0em 0.0em 0.0em 0.416667em; /* 0 0 0 5 */
    -fx-text-fill: -fx-text-background-color;
}
.check-box > .box {
    -fx-background-radius: 3, 2, 1;
    -fx-padding: 0.166667em 0.166667em 0.25em 0.25em; /* 2 2 3 3 */
`
```

6.2. Estilos mediante CSS en JavaFX

La sintaxis del CSS es igual que para Web. Sólo cambian las propiedades CSS.

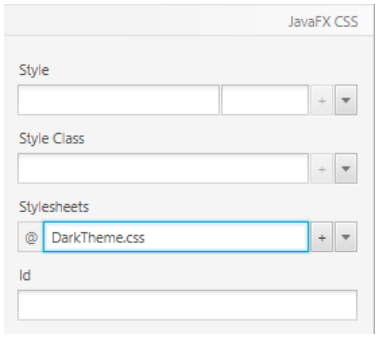
Debes conocer

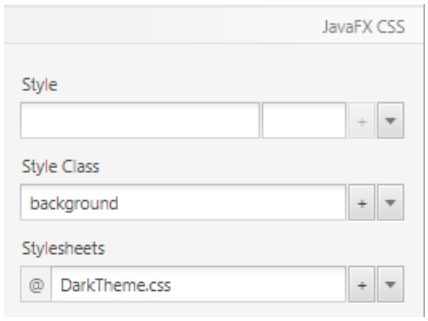
Referencia a la guía del CSS en JavaFX

Una o varias hojas de estilo puede ser aplicada a toda la escena o a un nodo (y sus hijos). Se podrá añadir por código, asignando una nueva hoja de estilo o directamente sobre el elemento:

button.setStyle("-fx-background-color: green;");

También se podrá añadir una hoja de estilo a través del Scene Builder. (abajo se muestra cómo se selecciona el archivo css y luego la clase concreta para una propiedad)

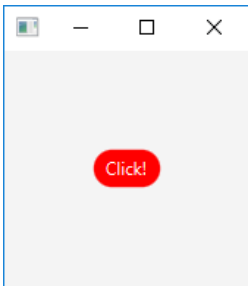




Ejemplo
Aquí un ejemplo sencillo en el que se crean nuevos estilos sobre el botón y se incorpora esa nueva hoja de estilo al proyecto.

```
// dentro del main
scene.getStylesheets().add(getClass().getResource("application.css").toExternalForm());

// dentro de application.css
.button {
-fx-background-color: red;
-fx-text-fill: white;
-fx-background-radius: 16px;
}
```

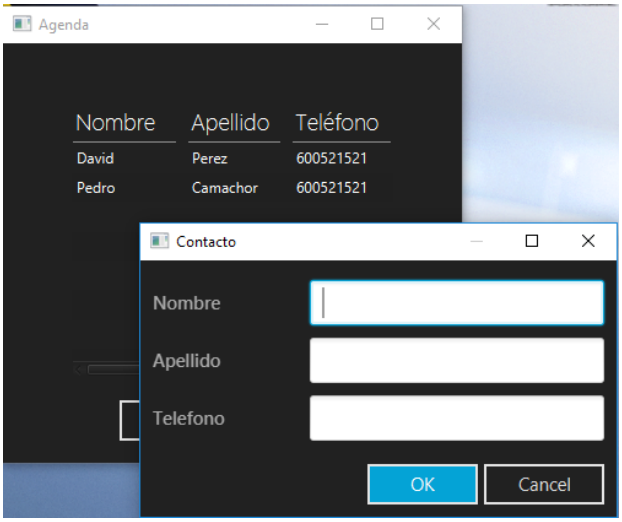


Puedes descargarte el proyecto de este ejemplo aquí.

Para saber más

Te facilitamos un archivo DarkTheme.css con el que podrás conseguir otra estética totalmente diferente a tu

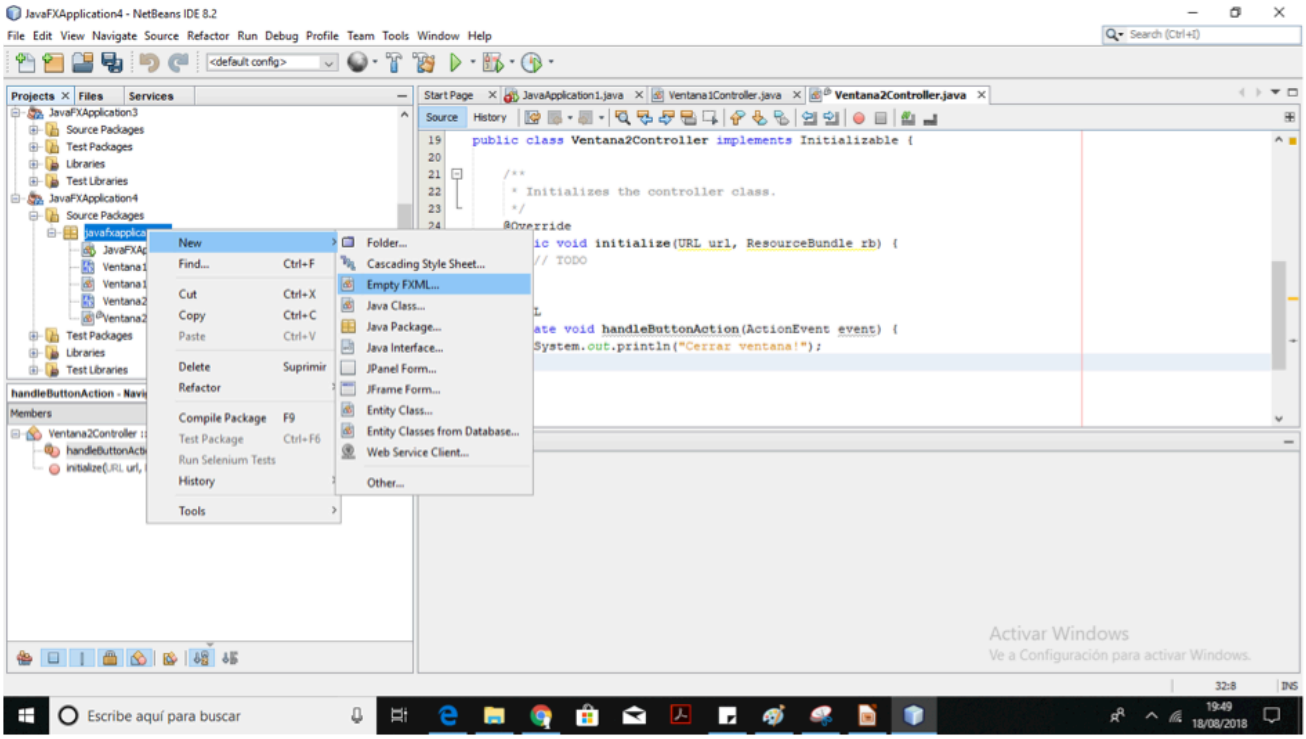
interfaz gráfico. ¡Pruébalo en alguno de los ejemplos y miniejercicios!



7. Ejemplo aplicación con varias ventanas

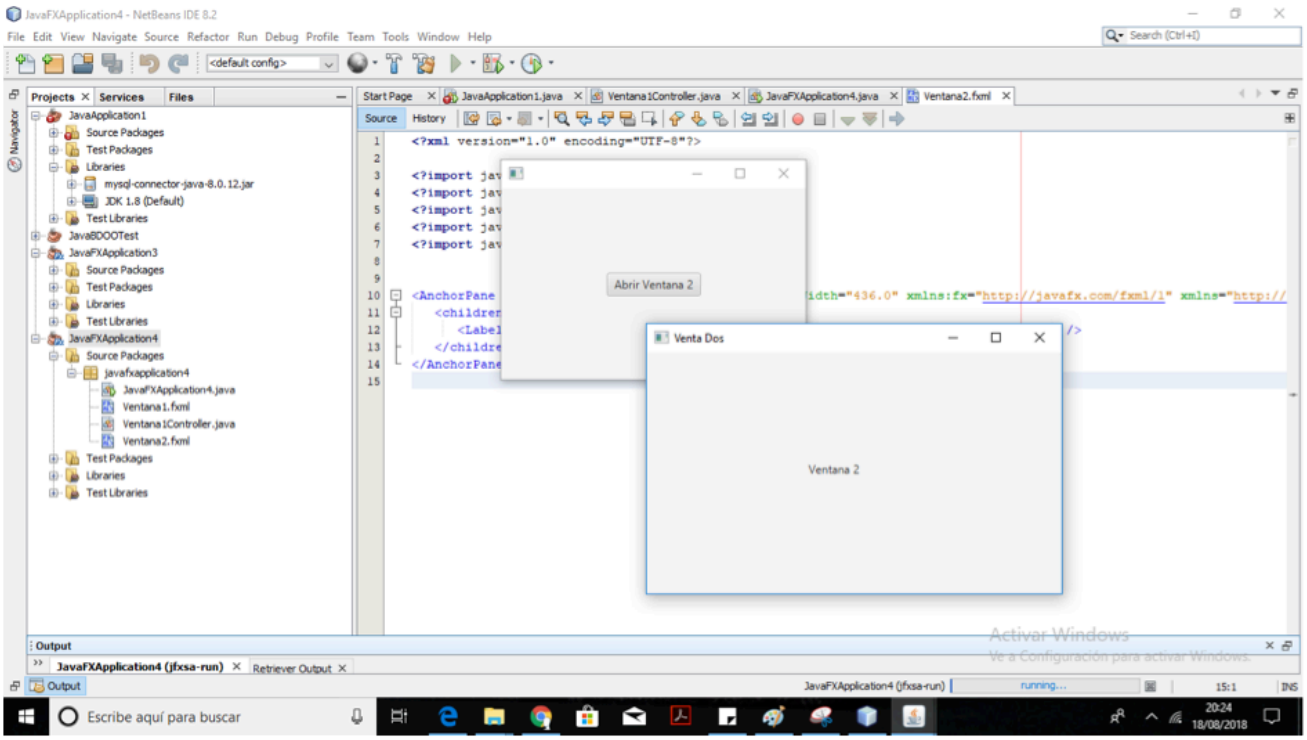
Aunque hemos avanzado muy rápido, creemos que debes ser capaz de hacer aplicaciones con un interfaz gráfico que tenga varios controles que generen eventos y construir una lógica de programa coherente. Antes de finalizar la unidad nos queríamos detener en un ejemplo que permita manejar dos ventanas FXML.

En esta situación tendremos que saber generar una ventana nueva desde NetBeans. Es tan sencillo como abrir un archivo nuevo como FXML vacío. Si esta opción no apareciera, deberás pulsar sobre *Other*.



En el código, lo importante será manejar las ventanas como objetos de la clase Stage, y los métodos show() y close() para mostrar y cerrar respectivamente. También están los métodos setVisible(true | false) para ocultar sin cerrar la ventana pero manteniéndose en memoria (+ info sobre la clase Stage).

Aquí os dejamos un ejemplo sencillo de cómo realizarlo.



Para saber más

En este post se muestra cómo hacer el mismo ejemplo pero usando el patrón MVC. Es en la clase principal donde se tienen los métodos para abrir las ventanas. La clase controladora de cada ventana lo que hace es recibir los eventos y manda la orden al método correspondiente en la clase principal. Para poder conseguir esto, hay que enlazar bien los objetos. Echa un vistazo al artículo y a los ejemplos.